



Audit Nexus / Gamebook

Projet : PushQuest **Module** : Nexus (subsystem gamebook) **Branche** : feat/gamification-pass-2 **Date** : 2026-05-27
Auditeur : Claude Opus 4.7 **Méthodologie** : Audit chirurgical sans complaisance, sans modification de code

Sommaire

1. [Cartographie — MAP APPLICATION.md](#)
2. [Tech Lead — AUDIT 01 CODE.md](#)
3. [DBA — AUDIT 02 DATABASE.md](#)
4. [Game Designer — AUDIT 03 GAMEDESIGN.md](#)
5. [Narrative Designer — AUDIT 04 NARRATIF.md](#)
6. [UX Designer — AUDIT 05 UX.md](#)
7. [QA Engineer — AUDIT 06 QA BUGS.md](#)
8. [Conformité / Sécurité — AUDIT 07 CONFORMITE.md](#)
9. [Questions ouvertes — AUDIT 08 QUESTIONS.md](#)
10. [Synthèse exécutive — AUDIT 00 SYNTHESE.md](#)

<div class="pagebreak"></div>



MAP_APPLICATION.md — Cartographie du module Nexus

Phase 0 — Audit chirurgical Nexus. Pas de modification de code.

1. Périmètre couvert

Le module **Nexus** (alias *gamebook*) regroupe :

Catégorie	Chemin racine	Volume
Composants UI client	src/app/gamebook/	36 fichiers, ~14 000 lignes
Routes API serveur	src/app/api/gamebook/	95 fichiers
Bibliothèques métier	src/lib/gamebook/	23 fichiers, ~9 700 lignes
Catalogue items	src/lib/gamebook/items.ts + src/config/badges.ts	2 fichiers
Routes admin liées	src/app/api/admin/rescue-player/	1 route
Migrations Prisma	prisma/migrations/ (filtre Nexus)	~22 migrations
Models Prisma	prisma/schema.prisma	4 modèles directs + 3 satellites
Scripts utilitaires	scripts/audit-doors.mjs	1 fichier

Hors périmètre mais utilisés en lecture : src/lib/xp.ts (XP_ANIMALS, getLevelDetails), src/lib/badges.ts (BADGE_DEFINITIONS, recalc), src/lib/challenge.ts (getTodayISO, daily reps), src/lib/rewards.ts .

2. Arborescence détaillée

2.1 Composants UI client — `src/app/gamebook/`

Point d'entrée Next.js

Fichier	Lignes	Rôle
<code>page.tsx</code>	30	Page Next.js <code>/gamebook</code> server-side. Récupère session + state initial + injecte <code><GamebookClient></code> .
<code>GamebookClient.tsx</code>	230	Wrapper SSR→client. Charge initial state, gère erreur d'hydratation, monte MapClient.

Orchestrateur monolithique

Fichier	Lignes	Rôle
<code>MapClient.tsx</code>	5791 ⚠️	Cœur applicatif client. Gère état joueur, mouvement, popups, cinematics, NPCs, broadcasts Pusher, tous les handlers <code>pressa</code> / <code>tryMove</code> , ouverture de TOUS les modals. C'est le monolithe critique à refactoriser.

Composants tile / sprite

Fichier	Lignes	Rôle
<code>TileCell.tsx</code>	1167	Rendu visuel CSS d'une tile (~60 cas dans un switch déguisé). Accepte <code>overlayEmoji</code> pour les cages véto.
<code>PlayerSprite.tsx</code>	122	Sprite joueur (corps + direction + lunettes).
<code>MontVentouxSideView.tsx</code>	309	Vue de profil pseudo-3D du Mont (parallax 3 couches, route inclinée, cycliste animé).
<code>PiaffiniFlightScreen.tsx</code>	156	Cinématique plein écran du vol PIAFFINI.
<code>BlackScreen.tsx</code>	66	Écran noir transitoire.
<code>FrozenScreen.tsx</code>	154	Overlay anti-cheat (gel quand suppression de reps).

Modales principales

Fichier	Lignes	Rôle
<code>StartMenu.tsx</code>	133	Menu START (SAC / DAEMON / RETOUR). Gating conditionnel sur <code>hasRecoveredDaemon</code> .
<code>InventoryModal.tsx</code>	397	Sac + actions sur items.
<code>ShopModal.tsx</code>	337	Shop générique paramétré par prop <code>shop</code> (10 vendeurs supportés).
<code>BibliothequeModal.tsx</code>	377	Bibliothèque Macaron/Muscuville avec topics navigables.
<code>TamagotchiModal.tsx</code>	538	Adoption + soin tamagotchi + boutons défis.
<code>DaemonTeamModal.tsx</code>	445	Équipe Daemon 6 slots + stats + items applicables (sérums/pierres).
<code>BattleModal.tsx</code>	517	Combat Pokémon-style (attaques/switch/sac/fuite).
<code>SaiyanLevelUpModal.tsx</code>	275	Répartition points stats après level-up Daemon.
<code>PlayerMapModal.tsx</code>	181	Mini-map mondiale (item map).

Fichier	Lignes	Rôle
TreeBookModal.tsx	109	Pokédex des arbres.
FastTravelModal.tsx	128	Voyage rapide entre villes (désactivé v3.33).
BestioleNamingModal.tsx	120	Nommer les bestioles à la 1 ^{re} rencontre.

Modales jeux de hasard

Fichier	Lignes	Rôle
CasinoModal.tsx	195	Roulette rouge/noir Bourg & Pépiteville.
CasinoPatternModal.tsx	278	Roulette pattern Muscuville.
CasinoPatternVegasModal.tsx	226	Variante VIP Vegas.
SlotMachineModal.tsx	117	Machine à sous Vegas.
LottoPouleModal.tsx	123	Grille 4×4 Vegas.
StopOuEncoreModal.tsx	183	Stop ou Encore Vegas.
CockfightModal.tsx	158	Combat de coqs Vegas.

Modales contexte Muscuville / Vegas

Fichier	Lignes	Rôle
ArenaModal.tsx	163	Arène Muscuville (4 champions + revanches).
VideurModal.tsx	132	Portier Arrabbiata du bar TB.

Modales Pastagone v4.0

Fichier	Lignes	Rôle
PastagoneCelluleModal.tsx	183	Interrogatoire CARBONE (3 défis réels pompes/gainage/squats).
PastagoneInfirmierieModal.tsx	94	FUSILLI heal 50 reps / 3 fois/jour.
PastagoneCuisineModal.tsx	167	RIGATONI shop + énigme BOLOGNION.
PastagoneArmurerieModal.tsx	130	PESTO Jr wearables Daemon.
PastagoneBriefingModal.tsx	194	TAGLIA récap arc + défi boss + choix orphelin.
PastagoneTourModal.tsx	188	Tour de Garde (25 PNJ rotation + cooldown 30s).

2.2 Routes API serveur — `src/app/api/gamebook/`

95 routes, regroupées par domaine.

État global & technique (6)

Route	Verbe	Rôle
state/route.ts	GET / POST / DELETE	Source de vérité. GET retourne tout l'état joueur + autobootstrap créateur + auto-migration daemon. POST sauvegarde position/flags. DELETE reset.
players/route.ts	GET	Polling des autres joueurs visibles sur la même map.
broadcast/route.ts	POST	Diffuse events Pusher (cinematic:trigger).

Route	Verbe	Rôle
pusher-auth/route.ts	POST	Auth Pusher.
spend/route.ts	POST	Dépense d'énergie ad-hoc (rarement appelée).
travel/route.ts	POST	Fast-travel entre villes.

Tamagotchi (legacy v3.14 → v4.0) (8)

Route	Rôle
tamagotchi/adopt	Adoption initiale chez V3T (snapshot vetFirstVisitLevel).
tamagotchi/feed	Nourrissage (déprécié au profit de feed-pates).
tamagotchi/feed-pates	Consomme corned_pates + heal happiness.
tamagotchi/drink	Consomme gourde + heal happiness.
tamagotchi/check-defis	Valide les 7 défis d'adoption.
tamagotchi/liberer	Officialise adoption (recovered=true) + badge animal totem.
tamagotchi/in-bag	Toggle "rangé dans le sac".
tamagotchi/turn-talk	Animal follower interactif (dialogues random).
tamagotchi/use-serum	Sérum intelligence (v3.24d — <i>attention au doublon avec daemon/use-serum</i>).
v3t/talk	Commentaire dynamique V3T (déprécié de fait par dernier commit).

Daemon v4.0 (14)

Route	Rôle
daemon/list	Liste équipe (jusqu'à 6 slots) avec stats enrichies.
daemon/public-list	Liste publique recovered=true (utilisée par cages véto).
daemon/reorder	Réordonne slots (set leader).
daemon/feed daemon/drink daemon/heal daemon/in-bag	Actions de base.
daemon/use-serum	Active unlockedAt (créateur ou Phase 4 mafia).
daemon/evolve	Pierre d'évolution (change type).
daemon/boost-stat	Sérum permanent (+5 stat).
daemon/equip-item daemon/unequip-item	Wearables.
daemon/allocate-points	Saiyan points → bonus[X].
daemon/battle/start daemon/battle/action daemon/battle/use-item	Combat.

Pastagone v4.0 (12)

Route	Rôle
pastagone/capture	Cinématique arrestation → cellule.

Route	Rôle
pastagone/interrog-defi	Validation 1 des 3 défis (pompes/gainage/squats).
pastagone/infirmier-heal	FUSILLI.
pastagone/cuisine-puzzle	Énigme BOLOGNION 3 steps.
pastagone/tour-rotate pastagone/tour-battle	Tour de Garde.
pastagone/coulter-battle pastagone/capolino-mid-battle pastagone/boss-battle	Mini-bosses + boss.
pastagone/faa-gift	One-shot + 100 reps.
pastagone/orphan-choose	Choix Anguillzap/Faucotron/Octopsy + ciné CAPOLINO fuite.
pastavegas/shoptower-climb	Stairs Tour Pullman.

Cinématiques & one-shots narratifs (10)

Route	Rôle
grant-bag	PEPITO / MAMAN offrent le sac.
grant-gym-energy grant-durum-energy	Récompenses one-shot reps.
piaffini/rescue	Sommet Tour → badge Sauveur.
mont/summit-reached	Sommet Mont → badge Conquérant.
bridge/route.ts	Pont Azuria + 5 PNJ + badge Pionnier + badge Star.
franss-joke	Easter egg Franss.
monstre/grant-amulette	Cadeau Monstre.
grass-sud-cutscene	Cutscene grass_sud.
nageur/defi	Défi Nageur.
painting/papa-boost	Tableau papa tour.

Shops (3)

Route	Rôle
shop/buy	Achat unifié pour 10 vendeurs (filtre par availableAt).
shop/info	Info contextuelle pour lastShopPurchase .
take-fruit	Cueillette arbres fruitiers (~10 types).

Inventaire (2)

Route	Rôle
inventory/use	Use d'item (corned_pates, gourde, baskets, etc.).
arrosoir/use	Arrosoir → fait repousser fruit.

Casinos (8)

Route	Rôle
casino/bet casino/croupier-talk casino/coin-found	Roulette Bourg/Pépité.
casino/pattern-spin casino/pattern-vegas-spin	Roulette pattern Muscuville/Vegas.
casino/slot casino/cockfight casino/lotto-poule casino/stop	Jeux Vegas additionnels.

Muscuville (4)

Route	Rôle
muscuville/champion	4 champions (1 ^{re} + revanche).
muscuville/interpellator-talk	Réceptionniste arène.
muscuville/rocks-pay	Rochers de sortie ouest (4000 reps modulés).
veto-muscu/heal	Vétérinaire Muscuville.
biblio-muscu/talk	Bibliothécaire (donne TreeBook).
arena/fight	Combat champion exo.

Macaron'île (3)

Route	Rôle
biblio/gift-tree-book	Don du TreeBook (déplacé Muscuville).
jardinier/talk jardinier/check	BASILICO mission fruits.
ornithologue/talk	+50 reps si animal=oiseau.

Team Boulette (Vegas) (7)

Route	Rôle
tb/videur tb/sbire tb/jamie tb/pesto tb/brute	Sous-fifres.
tb/validate-challenge	Défis du bar.
tb/boss	Boss IL CAPO.

Divers (5)

Route	Rôle
water/attempt water/push water/jojo-push	Canal coopératif.
bestiole/encounter	Hautes herbes du sud.
tower/climb	Tour des Pâtes Aiguës (gate exercices).
tree/discover	Découverte Pokédex arbres.
team/captain-bonus	Bonus capitaine d'équipe.
contest/defi	Défis intersalle Muscuville.
luck/talk	LINGUINI chance.
hotel/sleep	Hôtel Vegas.
guigui/recharge	Recharge GUIGUI (compte test).

2.3 Bibliothèques métier — `src/lib/gamebook/`

Fichier	Lignes	Rôle
<code>engine.ts</code>	218	Logique de jeu pure (intro Monstre, transitions). À auditer pour clarifier vs <code>mapEngine</code> .
<code>mapEngine.ts</code>	578	Mouvement (<code>tryComputeMove</code>), tiles bloquantes, entrées bâtiments, <code>INTERIOR_ENTRY_POSITIONS</code> .
<code>maps.ts</code>	2812 ⚠️	TOUTES les maps (32 maps construites in-code) + buildings + signs. Énorme monolithe.
<code>npcs.ts</code>	2565 ⚠️	TOUS les NPCs (~80+). Données + dialogues + interactions. Difficile à maintenir.
<code>items.ts</code>	1018	Catalogue items + capabilities + filtres shops.
<code>inventory.ts</code>	223	Parse/add/remove/hasIntactItem. Pur.
<code>dialogue.ts</code>	?	Dialogues longs (cinématiques, monologues).
<code>daemon.ts</code>	?	Stats Daemon, formules XP, <code>computeMaxHp</code> , Saiyan.
<code>battleState.ts</code>	?	Transition combat (apply player action, status, vitesse).
<code>attacks.ts</code>	?	~30 attaques + STRUGGLE.
<code>combat.ts</code>	?	Formule dégâts Gen 1, types matchups 10×10.
<code>pastagoneTourNpcs.ts</code>	175	Pool 25 PNJ Tour de Garde.
<code>tamagotchi.ts</code>	388	Logique tamagotchi (display, défis, recovered).
<code>happinessChanges.ts</code>	71	Deltas happiness (decay, gains).
<code>energy.ts</code>	137	Calcul <code>availableEnergy</code> côté serveur.
<code>creator.ts</code>	96	Bootstrap auto compte créateur.
<code>antiCheat.ts</code>	?	Gel gamebook si suppression de reps.
<code>difficulty.ts</code>	?	Ratio onboarding (jeunes joueurs paient moins).
<code>ratio.ts</code>	177	Probablement <code>applyRatio</code> (rabais).
<code>userLevel.ts</code>	97	Map level XP global → niveau Pokémon.
<code>teams.ts</code>	57	Captain bonus, équipes.
<code>casino.ts</code> <code>casinoBoost.ts</code>	?	Calculs roulette + boost LINGUINI.

2.4 Configuration

Fichier	Rôle
<code>src/config/badges.ts</code>	11 badges Nexus (parmi tous les badges PushQuest).
<code>src/lib/xp.ts</code>	XP_ANIMALS (100 entrées level→emoji) + <code>getLevelDetails</code> .
<code>src/lib/challenge.ts</code>	<code>getTodayISO</code> , daily target, ratios.
<code>src/lib/rewards.ts</code>	XP formules globales (capture mensuelle).

2.5 Migrations Prisma — `prisma/migrations/`

35 migrations au total, dont **22 liées à Nexus** (préfixe `*_v3_*` ou `*_v4_*`) :

Date	Migration	Sujet
20260520	add_gamebook_progress	Initial GamebookProgress
20260521	v3_map	Champs map
20260522	v3_1_route1	Route 1
20260524	v3_23c_conquerant / v3_23c2_contest_defis / v3_24a_teams / v3_24b_casino_pattern / v3_3_npcs	Plusieurs ajouts simultanés
20260525	v3_23e_franss_joke / v3_23f_bonus_surplus / v3_23g_water_attempts / v3_23h_vet_first_visit / v3_4a_energy_spent	Polish
20260526	v3_6_anti_cheat / v4_0_active_battle / v4_0_daemon_unlocked / v4_0_pending_stat_points / v4_pastagone_progress	v4.0 day 1
20260527	v4_capolino_mid_fuite / v4_pullman_coulter	v4.0 day 2
20260528-20260608	pepiteville / fruits_boots / tower / piaffini / water / tamagotchi / luck / polish / bestioles / casino / fast_travel	Suite historique

⚠ **Note** : Les dates de migration `20260528+` ont des préfixes datés **après** les migrations `20260520-20260527`. C'est cohérent dans l'ordre alphabétique car le suffixe `vX_Y_*` ordonne par version logique, mais le format date suggère une réalité historique différente. À vérifier en phase 1.2.

2.6 Scripts utilitaires

Fichier	Rôle
<code>scripts/audit-doors.mjs</code>	Vérifie <code>INTERIOR_ENTRY_POSITIONS</code> + <code>exitTargets</code> vs <code>tile walkability</code> . Réutilisable.

3. Points d'entrée

3.1 Route page Next.js

- `/gamebook` → `src/app/gamebook/page.tsx` → `<GamebookClient>` → `<MapClient>`.

3.2 Routes API consommées par MapClient (extraits)

- `GET /api/gamebook/state` — load au mount + refresh régulier.
- `POST /api/gamebook/state` — chaque mouvement, chaque set de flag.
- `GET /api/gamebook/players` — polling autres joueurs (~10s).
- `POST /api/gamebook/broadcast` — events Pusher.
- ~80 autres endpoints invoqués selon interaction (talk NPC, shopCounter, etc.).

3.3 Déclencheurs serveur-side

- `ensureCreatorBootstrap` (state route ligne 56) — auto-flag piaffini/swim/swim_set pour comptes isSystem.
- `ensureDaemonForTamagotchi` (state route, via daemon.ts) — auto-création Daemon depuis tamagotchi Json.
- Auto-set** `vetFirstVisitLevel` (state route ligne 117-135) — snapshot level à la 1^{re} visite véto.
- Tracking** `totalGamebookSteps` (state POST) — incrémenté à chaque mouvement.

- **Bootstrap badges PIAFFINI** (state route ligne 95-110) — auto-cr  ation XpAdjustment + BadgeEvent post-rescue.

3.4 Real-time

- **Pusher** : event `cinematic:trigger`, broadcast position joueurs. Channel `gamebook-{userId}`.

4. D  pendances inter-modules

4.1 Auth (next-auth)

- Toutes les routes API : `getSession(authOptions)` + extraction `userId`.
- Pas de session = 401.

4.2   nergie globale

- **Source de v  rit  ** : `ExerciseSet` (table all-time) → `getTodayReps(userId)` somme PUSHUP+SQUAT+PLANK/5+autres.
- **  nergie d  pens  e** : `GamebookProgress.energySpentToday` (reset minuit) + `bonusSurplus` (boost).
- **  nergie disponible** : `max(0, todayReps - energySpentToday + bonusSurplus)`.
- **Padding cr  ateur** : `padAvailableEnergyForCreator()` impose floor 1000 reps.
- **Pas de transaction stricte** entre `ExerciseSet.reps` et `GamebookProgress.energySpentToday`.

4.3 Badges / XP syst  me global

- `BadgeEvent` (UNIQUE_AWARDED) → trigger recalc via `lib/badges.ts`.
- `XpAdjustment` (table partag  e avec tout PushQuest) — r  affich  e dans timeline globale.
- 11 badges Nexus d  clar  s (cf. audit XP pr  c  dent).

4.4 Catalogues partag  s

- `XP_ANIMALS` (`lib/xp.ts`) — mapping level→animal utilis   partout (tamagotchi display, daemon emoji, cages v  to).
- `getUserLevelForGamebook` (`lib/gamebook/userLevel.ts`) — niveau Pok  mon depuis XP global PushQuest.

4.5 Pusher (real-time)

- `lib/pusher-server.ts` + `lib/pusher-client.ts` (hors gamebook).
- Channel par utilisateur (broadcast cinematic).

5. Mod  les Prisma utilis  s par Nexus

5.1 Mod  les directs (4)

Mod��le	R��le	Champs critiques	Volume actuel (DB locale)
GamebookProgress	Un row par user (composite <code>userId+chapterId="map_v3"</code>). Position + 60+ flags	<code>mapId/posX/posY</code> , <code>hasBag</code> , <code>tbBossBeaten</code> , <code>pastagone*</code> ,	~7 rows

Modèle	Rôle	Champs critiques	Volume actuel (DB locale)
	narratifs + Json (tamagotchi, inventory, activeBattle).	tamagotchi, inventory, activeBattle	
Daemon	1-N par user (slot 1-6 unique). Stats + bonheur + équipement + état combat.	speciesLevel, type, combatLevel, baseFor...baseEnd, currentHp, happiness, unlockedAt, equippedItems, attacksKnown	~3 rows
ExerciseSet	Toutes les reps encodées (PUSHUP/SQUAT/PLANK/PULLUP/etc.).	exercise, reps, date, userId	massif
XpAdjustment	Toutes les attributions d'XP.	userId, amount, reason, date	massif

5.2 Modèles satellites utilisés

Modèle	Lien Nexus
User	id, nickname, isSystem, isTester (gating créateur).
BadgeEvent	UNIQUE_AWARDED pour les 11 badges Nexus.
SpecialWorkoutEntry	Pas direct, mais WOD défis à la carte (mémoire séparée).
GlobalConfig	lastShopPurchase (info contextuelle shop).

5.3 Modèles totalement hors-périmètre (mais coexistent)

FineRecord, MonthlyChallengeEntry, MedicalCertificate, PotEvent, Bet/BetEntry/BetEvent/BetEventLike/BetResult, CoinAdjustment, QuotaModifier, BadgeOwnership, BadgeDefinition, BadgeLike, BadgeEventLike, WallMessage, UserStatus, StatusLike.

5.4 Relations

- **User 1 → 1 GamebookProgress** (via composite (userId, chapterId)).
- **User 1 → N Daemon** (slot unique par user).
- **User 1 → N ExerciseSet**.
- **User 1 → N XpAdjustment**.
- **User 1 → N BadgeEvent** (toUserId).

6. Observations préliminaires (sans complaisance)

6.1 ● Symptômes architecturaux préoccupants

1. **MapClient.tsx à 5791 lignes** = monolithe critique. Tout le client passe par là. Tout bug latent vivra ici. **À fragmenter avant tout autre refactor.**
2. **maps.ts à 2812 lignes** = monolithe données. Toutes les maps construites in-code, pas de séparation.
3. **npcs.ts à 2565 lignes** = même problème, ~80+ NPCs avec dialogues inline.
4. **Combinaison MapClient + maps + npcs ≈ 11 000 lignes** dans 3 fichiers. Le toucher = risque de régression sur l'ensemble du jeu.

6.2 ● Coexistence systèmes legacy/v4.0

- `tamagotchi` (Json sur GamebookProgress) ET `Daemon` (table relationnelle) cohabitent. L'auto-migration `ensureDaemonForTamagotchi` synchronise unilatéralement. **Double source de vérité = bug possible.**
- 2 routes `use-serum` : `tamagotchi/use-serum` (v3.24d sérum intelligence) et `daemon/use-serum` (v4.0 sérum Poussière). **Confusion possible.**

6.3 ● Migrations à date incohérente

Les migrations `20260528_*` jusqu'à `20260608_*` ont des dates **postérieures** aux migrations `20260526_v4_*` et `20260527_v4_*` qui ajoutent des champs récents (Pastagone, Coulter). Or alphabétiquement, `20260528_pepiteville` s'applique APRÈS `20260527_v4_pullman_coulter`. Si on regarde la sémantique, Pépiteville est *plus ancien* dans le jeu. **À auditer en phase 1.2 (database).**

6.4 ● Routes redondantes potentielles

- `grant-bag` vs `cinematic pepitoBag` : qui fait quoi ?
- `tamagotchi/feed` vs `tamagotchi/feed-pates` : la première est-elle morte ?
- `v3t/talk` désactivé de facto par le dernier commit (V3T NPC ouvre direct le modal) : à supprimer ou conserver ?

6.5 ● Trace narrative pas centralisée

Aucun fichier `flows.ts` ou `progression.ts` qui décrirait le parcours joueur. Le narratif est éparpillé : flags sur GamebookProgress, dialoguesAfterMacaronAwakened en NPC props, conditions inline dans MapClient. **Difficile d'auditer la cohérence narrative sans tout lire.**

6.6 ● Pas de tests automatisés détectés

Aucun fichier `*.test.ts` ou `*.spec.ts` dans le périmètre Nexus. Tout repose sur le test manuel. Pour un module de cette complexité, c'est un risque élevé.

6.7 ● Volumétrie données client

À chaque GET `/api/gamebook/state`, le serveur renvoie ~80 flags + inventory Json + tamagotchi Json + activeBattle Json. **Aucune pagination, aucune normalisation, aucune compression.** À considérer pour la performance mobile.

7. Statistiques globales

Indicateur	Valeur
Composants UI	36
Routes API	95
Modules libs	23
Modales	26 (sur 36 composants UI)
Routes serveur tamagotchi (legacy)	10
Routes serveur Daemon (v4.0)	14
Routes Pastagone v4.0	12
Migrations Prisma Nexus	22
Maps construites	32

Indicateur	Valeur
NPCs définis	~80
Items au catalogue	~30
Lignes totales périmètre Nexus	~24 500

● Fin de Phase 0

J'ai terminé MAP_APPLICATION.md. Pas de modification de code apportée.

Avant de passer à Phase 1.1 (AUDIT_01_CODE.md — Tech Lead / Audit Code & Architecture), dis-moi :

1. **Valide-tu le périmètre** (le module Nexus tel que cartographié, hors challenges quotidiens / classements / défis à la carte) ?
2. **Faut-il que j'ouvre chaque lib individuellement** pour préciser les rôles non clairs (engine.ts, dialogue.ts, daemon.ts, antiCheat.ts, etc.) avant d'attaquer l'audit code, ou tu fais confiance aux rôles inférés du nom ?
3. **Question méthodo** : pour l'audit code (phase 1.1), je vais devoir lire profondément les gros fichiers (MapClient 5791 lignes, maps 2812, npcs 2565). Tu préfères que je délègue cette lecture à un sous-agent `Explore` pour aller plus vite, ou je lis moi-même méthodiquement (plus lent mais plus précis) ?

J'attends ton go pour démarrer Phase 1.1.

<div class="pagebreak"></div>

Données collectées. Voici le rapport :

AUDIT_01_CODE.md — Tech Lead Silicon Valley QA

Phase 1.1 — Audit Code & Architecture. Sans complaisance, sans modification de code.

0. Résumé exécutif

Le module Nexus tient debout par habitude et par chance, pas par design. La séparation client/serveur existe sur le papier mais le client est un god-component de 5791 lignes qui dépend de tout, et le serveur n'utilise **aucune** transaction Prisma sur 95 routes mutatives. La sécurité de typage est volontairement contournée 292 fois (`prisma as any`). Aucun test automatisé. Code mort identifié dans 9 endroits (1 lib + 8 routes), `engine.ts` à 218 lignes n'est référencé NULLE PART.

Verdict global : ⚠ **Dette technique majeure**. Le code livre la valeur business demandée mais chaque ajout coûte exponentiellement plus cher. La prochaine refonte structurelle est non-optionnelle si la roadmap continue de s'étendre.

1. Indicateurs transverses (chiffres bruts)

Métrique	Valeur	Verdict
(<code>prisma as any</code>) (bypass typage)	292	● Massif

Métrique	Valeur	Verdict
<code>as any</code> (autres casts)	10	● Marginal
Casts structurels <code>as { ... }</code>	592	● Le typage explicite n'est pas utilisé
<code>console.warn/error/log</code>	80	● OK pour debug, à logger structuré en prod
<code>catch { /* silent */ }</code>	31	● Bugs invisibles
TODO/FIXME/HACK	0	● Bizarre (signe que personne ne note la dette)
Transactions Prisma <code>\$transaction</code>	0	● Aucune atomicité sur 95 routes mutatives
AbortController côté client	0	● Fetch fantômes possibles
useEffect dans MapClient	11	●
useState dans MapClient	57	● God component
<code>try { ... }</code> dans MapClient	88	● Idem
<code>fetch("/api/gamebook...")</code> dans MapClient	94	● Idem
<code>state.mapId === "..."</code> dans MapClient	85	● Gating spaghetti
Tests automatisés Nexus	0	●

2. Architecture globale — séparation des responsabilités

2.1 ● Le god component : `MapClient.tsx`

- **5 791 lignes** dans un seul composant React.
- **57 hooks** `useState` locaux → la moitié de l'état applicatif vit dans un seul composant.
- **94 appels** `fetch` vers 50+ routes différentes, mélangés inline dans des handlers JSX.
- **85 branches** `state.mapId === "..."` qui transforment ce composant en machine à états déguisée.
- **88 blocs** `try` dispersés, dont aucun avec stratégie de retry.
- Importe ~50 modules (modals, NPCs, items, map data).

Verdict : Ce composant viole `single-responsibility`, `open/closed`, et la moindre règle de cohésion. Toute modification non-triviale a un risque de régression imprévisible.

Recommandation : Découper en au minimum :

- `useGamebookState()` — hook custom qui gère état joueur (position, énergie, flags) + sync serveur.
- `useNpcInteraction()` — hook qui gère le `pressA` sur NPCs avec un router par `npcId`.
- `useTileInteraction()` — hook qui gère le `pressA` sur tiles (shopCounter, doorMat, animalCage, etc.).
- `useCinematicMachine()` — state machine dédiée aux cinématiques.
- `useBattleFlow()` — gestion BattleModal + state activeBattle.
- `<MapRenderer>` — composant pur de rendu de la grille et des sprites.
- `<ModalHost>` — un seul host qui mount/unmount tous les modals selon état.

2.2 ● Confusion `engine.ts` vs `mapEngine.ts`

Fichier	Lignes	Rôle déclaré	Rôle effectif
<code>engine.ts</code>	218	"Moteur du Gamebook. Fonctions pures."	Zéro importeur. Code mort intégral.

Fichier	Lignes	Rôle déclaré	Rôle effectif
<code>mapEngine.ts</code>	578	Pas commenté	Le VRAI moteur (tryComputeMove, isBlockingTile, INTERIOR_ENTRY_POSITIONS).

`engine.ts` est référencé uniquement depuis `src/_archive/gamebook_progress_v2/progress/route.ts` — du code explicitement archivé. C'est donc 218 lignes de code mort en `src/lib/gamebook/`.

Recommandation : supprimer `engine.ts` OU le renommer `_archive_engine.ts` si tu veux le garder par nostalgie.

2.3 Triple architecture des données de map

- `maps.ts` (2812 lignes) construit les tiles **en code impératif** via des `for` loops + assignements impératifs.
- `npcs.ts` (2565 lignes) déclare ~80 NPCs en **objets littéraux inline** avec dialogues hardcodés.
- `dialogue.ts` contient encore d'autres dialogues longs séparés des NPCs.

→ Pour ajouter un NPC, il faut toucher 1-3 fichiers, espérer ne pas casser le typage, et tester manuellement. Pas de schéma de données, pas de validation, pas de runtime check.

Recommandation : extraire les NPCs en JSON typé (`src/data/npcs/{map}.json`) + un loader qui valide via Zod au démarrage.

2.4 Aucun découpage logique côté serveur

- 95 routes API, chacune avec son fichier `route.ts` (Next.js App Router le force) — OK.
- **Mais** : aucune couche service partagée. Chaque route réimplémente :
 - Auth check (`getServerSession` + extraction `userId`) — 95×.
 - Lecture `GamebookProgress` — ~80×.
 - Cast `prisma as any` — 292×.
 - Logique métier mélangée avec persistance.

Recommandation : créer `src/lib/gamebook/services/` avec :

- `progressService.ts` — read/update `GamebookProgress`.
- `energyService.ts` — calcul/débit/crédit énergie.
- `daemonService.ts` — list/create/update daemons.
- `inventoryService.ts` — déjà partiellement présent dans `inventory.ts` mais pas utilisé partout.
- `xpService.ts` — création atomique de `BadgeEvent` + `XpAdjustment`.

3. Anti-patterns récurrents

3.1 (prisma as any) × 292

Le typage Prisma est contourné systématiquement. Causes probables :

1. Le client Prisma n'a pas été régénéré après chaque migration (cf. règle d'or memo : "EPERM Windows lock sur prisma generate").
2. Les nouveaux champs (Daemon, pastagone*, etc.) sont annotés mais le type généré ne reflète pas la DB.
3. Mode "ça marche, on bouge".

Conséquence concrète : aucune erreur de compilation si on tape un mauvais nom de champ (`pastagonneArrested` au lieu de `pastagoneArrested`). Le bug arrive en runtime.

Recommandation : forcer `prisma generate` après chaque migration via un hook `postinstall` ET supprimer tous les `as any` Prisma en 3 PR (par domaine).

3.2 ● Dynamic imports partout dans les routes API

```
const { applyHappinessDelta, HAPPINESS_DELTAS } = await import("@/lib/gamebook/happinessChange");
```

Pattern présent dans 20+ routes serveur. Justification présumée : éviter les cycles d'import OU lazy-load. En réalité :

- ❌ Désactive le tree-shaking — tout est bundlé runtime.
- ❌ Cache l'arbre de dépendances aux IDE → impossible de "find usages" en static.
- ❌ Coût runtime à chaque appel de route (résolution module).
- ✅ Le seul cas légitime serait éviter un cycle, mais aucun cycle réel n'est documenté.

Recommandation : convertir tous ces dynamic imports en imports statiques en haut du fichier. Si vrai cycle détecté → refactor.

3.3 ● 31 } catch { /* silent */ }

Catalogue des silent catches :

```
src/app/gamebook/MapClient.tsx          13 X
src/app/gamebook/DaemonTeamModal.tsx    3 X
src/app/gamebook/BattleModal.tsx        2 X
src/app/gamebook/PastagoneCelluleModal.tsx 2 X
... (10+ autres modals)
```

Pour chaque appel `fetch` qui échoue silencieusement, le joueur ne sait pas qu'il y a eu un problème (réseau, 500, 401 après expiration session). Le state local reste mais le serveur ne sait rien. Au prochain reload → divergence.

Recommandation : un wrapper `apiFetch(url, opts)` qui :

- Centralise headers/credentials.
- Catch 401 → redirect login.
- Catch 500 → toast "Erreur serveur, réessaie."
- Catch network → toast "Erreur réseau."
- Expose un log structuré.

3.4 ● Aucune transaction Prisma sur 95 routes mutatives

```
$ grep -rln 'prisma\.\$transaction' src/app/api/gamebook
# (vide)
```

Exemples de routes qui devraient être atomiques mais ne le sont pas :

- `/api/gamebook/shop/buy` : débit energySpent + write inventory + write totalShopSpend → 3 writes séparés.
- `/api/gamebook/mont/submit-reached` : badgeEvent + xpAdjustment + GamebookProgress.update → 3 writes.
- `/api/gamebook/pastagone/orphan-choose` : daemon.create + gamebookProgress.update → 2 writes.
- `/api/gamebook/daemon/battle/action` : daemon.update + gamebookProgress.update → 2 writes.

Conséquence concrète : si la première écriture réussit et la seconde échoue (Neon timeout, lock, deadlock), le joueur a l'XP sans le badge OU le daemon sans le slot mis à jour. Récupération manuelle obligatoire.

Recommandation : wrapper toutes les routes multi-write dans `prisma.$transaction()`. Coût trivial vs bénéfice critique.

3.5 ● useEffect avec deps incomplètes

Repéré dans MapClient (11 useEffect, plusieurs avec [] ou deps minimales alors que le hook utilise des state externes). Exemple flagrant :

```
useEffect(() => { ; (async () => {
  await refresh()
  setLoading(false)
})(); }, [])
```

Dans DaemonTeamModal.tsx : refresh() capture focused dans sa closure mais l'effect a [] comme deps. Si focused change, la version de refresh utilisée reste l'initiale. Fonctionnellement OK ici (l'utilisateur clique pour set focused, qui re-render et re-attache). Mais c'est fragile.

Recommandation : activer eslint-plugin-react-hooks avec règle exhaustive-deps en error.

3.6 ● Conditions inline gating (85 occurrences)

```
if (state.mapId === "veterinaire" && tile === "shopCounter") { setShowTamagotchi(true); return
if (state.mapId === "pastagone_cuisine" && tile === "foodBag") { ... }
if (state.mapId === "vegas_shoptower_1" || state.mapId === "vegas_shoptower_2" ...
```

Le pressA de MapClient est devenu un router de 600+ lignes avec ~85 conditions de gating. Ajouter une interaction nouvelle = trouver le bon endroit dans la chaîne, espérer ne pas casser une priorité existante.

Recommandation : table de routing (mapId, tile|npcId) → handler extraite. Permet ajout déclaratif sans toucher le god component.

4. Code mort, dupliqué, ou inutilisé

4.1 ● engine.ts — 218 lignes mortes

0 importeur (sauf _archive/). À supprimer.

4.2 ● 8 routes API zéro consumer client

Route	Statut
biblio/gift-tree-book	0 client. Probablement remplacée par tree-book à Muscuville.
daemon/equip-item / daemon/unequip-item	0 client. Routes créées mais aucune UI ne les appelle.
daemon/heal	0 client. Route présente, jamais appelée.
muscuville/interpellator-talk	0 client. Code mort ?
pusher-auth	0 client direct (peut être appelée par lib pusher-client côté Pusher SDK). À valider.
tamagotchi/use-serum	0 client. Doubleton avec daemon/use-serum ?
v3t/talk	0 client. Dernier commit a désactivé l'appel. À supprimer.

Recommandation : pour chaque route, soit la connecter à une UI, soit la supprimer. Le placebo "ça pourrait servir" coûte plus que ça ne rapporte.

4.3 ● Routes redondantes / doublons

Route #1	Route #2	Verdict
tamagotchi/feed	tamagotchi/feed-pates	feed est probablement obsolète.
tamagotchi/use-serum (v3.24d sérum intelligence)	daemon/use-serum (v4.0 sérum Poussière)	Confusion sémantique forte. À renommer/supprimer.
grant-bag	setCinematic("pepitoBag") côté client	Qui de l'API ou du cinematic effectue le grant ? À tracer.

4.4 ● Dossier _archive/ dans src/

```
src/_archive/gamebook_progress_v2/progress/route.ts
src/_archive/sanctuaire_v1/fsm_dialogue.json
src/_archive/sanctuaire_v1/GamebookClient.tsx
src/_archive/sanctuaire_v1/page.tsx
src/_archive/sanctuaire_v1/SanctuaireTab.tsx
```

Le code mort archivé pollue les recherches grep/IDE. **Recommandation** : déplacer hors de `src/` (par exemple `archive/`) ou supprimer (git history le conserve).

4.5 ● Pattern dupliqué : “battle setup”

`/api/gamebook/daemon/battle/start`, `/api/gamebook/pastagone/tour-battle`,
`/api/gamebook/pastagone/coulter-battle`, `/api/gamebook/pastagone/capolino-mid-battle`,
`/api/gamebook/pastagone/boss-battle` : 5 routes qui font toutes la même chose (lire leader, calculer effective
stats avec snapshot wearables, créer BattleState).

~80 lignes dupliquées par fichier × 5 = 400 lignes dupliquées.

Recommandation : extraire un helper `buildBattleStart(userId, enemyDescriptor) → BattleState`.

5. Gestion d'erreurs & cas limites

5.1 ● Pas de retry, pas d'abort, pas de timeout

- **0 AbortController** côté client → fetch en cours pendant qu'on navigue ou ferme un modal continue à muter le state local après unmount → warnings React + race conditions.
- Pas de retry sur les routes critiques (battle action, shop buy).
- Pas de timeout explicite (le runtime Vercel a son propre timeout).

5.2 ● Validation d'entrée serveur inégale

Échantillon de routes :

Route	Validation body	Verdict
/state POST	Inputs validés (mapId in list, posX/posY 0-30, direction in list)	✓
/shop/buy	itemKey string check	● Pas de check explicite que l'item est buyable.
/daemon/battle/action	body.action.kind check	● Pas de schéma global, juste switch.

Route	Validation body	Verdict
/pastagone/orphan-choose	orphan in dict	✓
/daemon/allocate-points	Allocation parsée via clampInt	✓

Recommandation : un schema Zod par route, validation strict en début de handler. Sortie type-safe.

5.3 🟡 Null safety à la louche

```
const userRow = await (prisma as any).user.findUnique({...})
const isCreator = userRow?.isSystem === true // ✓ optional chaining OK
// ...
const tam = p.tamagotchi as { name?: string; currentLevel?: number; ... }
if (!tam.name || typeof tam.currentLevel !== "number") return // ✓
```

Mais ailleurs (MapClient) :

```
const npcInFront = npcsWithPos.find(...)
if (npcInFront) { ... const npcId = npcInFront.npc.id // 🟡 npc.id non-typé garanti
```

Et :

```
const tile = map.tiles[ny][nx] // 🟡 si y/x hors map → exception runtime
```

`tryComputeMove` valide les bounds en amont, donc OK ici. Mais ailleurs dans MapClient (les overrides), pas toujours.

5.4 🔴 État local non-synchronisé serveur

Pattern observé :

```
setState((s) => ({ ...s, pastagoneEscaped: true }))
// PUIS plus tard : POST /state qui ne renvoie pas pastagoneEscaped
```

Le client met à jour son state localement *avant* que le serveur ait confirmé. Si le serveur 500, l'état local est divergent. Récupéré uniquement à la prochaine GET `/state` complète.

Risque : un utilisateur qui ferme l'app pendant que son `pastagoneEscaped=true` local est en attente de sync server perd la cohérence.

6. Performance

6.1 🟡 GET /api/gamebook/state retourne tout

Réponse type :

- `state` (80+ champs flat, incluant Json tamagotchi + Json activeBattle + Json inventory + Json visitedTowns)
- `todayReps`, `energySpentToday`, `bonusSurplus`, `availableEnergy`
- `frozen`, `frozenUntil`, `inventory`, `hasBag`, `fruitsTaken`, `towerFloorReached`, `difficultyRatio`
- `tamagotchi` (Json reformaté)
- `isCreator`

→ ~5-15 KB par appel, appelé au mount, après chaque mouvement (POST + GET refresh), et à la fermeture des modals. Mobile en 3G : douloureux.

Recommandation :

- Séparer endpoints : `/state/position` (rapide, juste mapId+pos+direction), `/state/full` (au mount uniquement).
- Cache client (SWR/React Query) avec invalidation ciblée.

6.2 ● Pas de cache `/players`

`/api/gamebook/players` est polled (~10s) pour afficher autres joueurs visibles. Aucun cache HTTP, aucun cache mémoire, aucun stale-while-revalidate.

6.3 ● Maps construites à chaque import

`maps.ts` construit les 32 maps à l'import via fonctions `buildXxx()`. C'est OK car module-level (1 fois par worker), mais ça gonfle le cold start serverless.

Recommandation : sérialiser une fois en JSON statique.

7. Dette technique récapitulée

Item	Sévérité	Impact effort
Aucun test automatisé Nexus	●	Régression invisible à chaque commit
God component MapClient	●	Toute évolution = risque cascade
<code>(prisma as any) × 292</code>	●	Bug typage en runtime
0 transaction Prisma	●	Corruptions partielles possibles
Silent catches × 31	●	Bugs invisibles
Dynamic imports systématiques	●	Tree-shaking off, perf cold start
Code mort (engine + 8 routes)	●	Pollution / confusion
<code>_archive/</code> dans <code>src/</code>	●	Pollution recherches
Pas de schema validation Zod	●	Validation manuelle inégale
Pas d'AbortController client	●	Memory leaks possibles
5 routes battle/setup dupliquées	●	400 lignes dupliquées
Pas de retry strategy fetch	●	UX dégradée sur réseau instable

8. Top 10 bombes à désamorcer (prio Tech Lead)

#	Bombe	Effort	Impact
1	Ajouter <code>prisma.\$transaction()</code> sur les 10 routes les plus mutatives	M	●
2	Wrapper <code>apiFetch</code> centralisé + retry + abort	M	●
3	Découper MapClient en 6-8 hooks customs + ModalHost	L	●
4	<code>prisma generate</code> enforced post-migration + suppression des <code>(prisma as any)</code>	M	●
5	Schema validation Zod sur les 20 routes critiques	M	●

#	Bombe	Effort	Impact
6	Test E2E sur le golden path (Bourg → Macaron → Pastagone)	L	
7	Supprimer <code>engine.ts</code> + 8 routes mortes + <code>_archive/src/</code>	S	
8	Convertir dynamic imports en static imports	S	
9	<code>eslint-plugin-react-hooks</code> exhaustive-deps en error	S	
10	Snapshot <code>maps.ts</code> en JSON statique pré-build	M	

S = small (< 1 jour), M = medium (1-3 jours), L = large (1-2 semaines).

9. Notes pour les phases suivantes

- **Phase 1.2 (DB)** doit creuser l'absence de transactions, le timing des migrations (`20260528_*` dates incohérentes), et les race conditions XP/énergie.
- **Phase 1.4 (Narratif)** doit lister tous les flags de progression pour cartographier les flows.
- **Phase 1.6 (QA)** doit ouvrir une issue pour chaque silent catch + chaque route morte.
- **Phase 1.7 (Conformité)** doit tracer chaque `as any` à la consigne d'origine — combien sont des contournements documentés vs des paresse.
- Ce que je n'ai PAS audité ici (volontairement, c'est ailleurs) :
 - Les schémas Prisma eux-mêmes (Phase 1.2).
 - Les flows narratifs end-to-end (Phase 1.4).
 - Les bugs visuels (Phase 1.5).
 - La conformité aux consignes (Phase 1.7).

Fin de Phase 1.1 — AUDIT_01_CODE.md

Pas une ligne de code modifiée. Aucune correction appliquée.

Auto Mode actif → j'enchaîne sur Phase 1.2 (AUDIT_02_DATABASE.md — Database Admin) dans la réponse suivante.

Si tu veux que je marque une pause et que tu valides avant Phase 1.2, dis "stop". Sinon je continue.

<div class="pagebreak"></div>

Données complètes. Je rédige le rapport.

AUDIT_02_DATABASE.md — Administrateur Base de Données

Périmètre : Nexus / Gamebook subsystem

Branche : `feat/gamification-pass-2` (HEAD `6f0bb65`)

Date : 2026-05-27

Posture : Sans complaisance

Critères :  Bloquant prod •  Critique •  Moyen •  Mineur

1. Vue d'ensemble — Inventaire schéma

Métrique	Valeur	Verdict
Modèles Prisma déclarés	26	—
Tables effectivement créées par migrations	12	● 14 modèles fantômes
Champs sur <code>GamebookProgress</code>	130	● God-table
Champs sur <code>Daemon</code>	40	● Trop large
Migrations totales	34	—
<code>prisma.\$transaction</code> dans gamebook	0	● Aucune atomicité
<code>FOR UPDATE / pg_advisory_lock</code>	0	● Aucun verrouillage
Routes mutant <code>energySpentToday</code> sans tx	31	● Race condition systémique

2. Cohérence schéma Prisma

2.1 ● God-table `GamebookProgress` (130 champs)

Type	Count	Exemples
Boolean	42	<code>hasBag</code> , <code>treeObstacleCleared</code> , <code>piaffiniRescued</code> , <code>pastagoneBossBeaten</code>
Json / Json?	24	<code>flags</code> , <code>history</code> , <code>inventory</code> , <code>tamagotchi</code> , <code>activeBattle</code> , <code>pastagoneCuisinePuzzle</code>
String / String?	27	<code>mapId</code> , <code>phase</code> , <code>videurState</code> , <code>casinoBetsDate</code>
Int / Int?	22	<code>posX</code> , <code>posY</code> , <code>energySpentToday</code> , <code>luck</code> , <code>bonusSurplus</code>
DateTime / DateTime?	11	<code>createdAt</code> , <code>lastSeen</code> , <code>gamebookFrozenUntil</code> , <code>pastagoneInterrogStart</code>

Diagnostic : Anti-pattern « Single Massive Save Object ». Chaque interaction joueur (déplacement, achat, dialogue, combat) déclenche un `update` de **tous les champs concernés** sur une ligne unique par utilisateur. Le payload moyen d'un round-trip dépasse 8 KB. Impossible à partitionner, impossible à archiver, impossible à indexer finement.

Conséquences concrètes :

- Verrouillage pessimiste implicite : chaque écriture verrouille toute la ligne, donc toute concurrence sur le même `userId` se sérialise → latence monte quand le joueur poste plusieurs actions rapprochées.
- L'absence d'index secondaires sur les 42 booléens force des full-scans pour toute statistique ad-hoc (« combien de joueurs ont sauvé PIAFFINI ? »).
- Aucune séparation domaine : casino, combat, narration, inventaire, daemon, économie cohabitent dans la même ligne.

2.2 ● Anti-pattern « Date stockée en String »

Onze champs nommés `*Date` typés `String` au lieu de `DateTime` :

```
casinoBetsDate, lastLuckTalkDate, lottoPouleDate, stopOuEncoreDate,
cockfightDate, slotMachinesDate, casinoBoostDate, lastArenaDate,
vetoMuscuLastVisitDate, lastDailyDecayDate, happyFlowerLastDate,
lastHotelSleepDate, energySpentDate
```

Format attendu : `YYYY-MM-DD` (TZ Europe/Paris).

Risques :

- Toute comparaison se fait lexicographiquement — fonctionne pour ISO mais casse au moindre format alternatif.
- Impossible d'utiliser les opérateurs Prisma `gte` / `lt` avec sémantique calendaire (fuseaux, été/hiver).
- Migrations multi-pays impossibles sans rewrite.

2.3 14 modèles déclarés mais jamais migrés (schema drift)

Les `CREATE TABLE` dans `prisma/migrations/*/migration.sql` couvrent **12 tables** seulement :

✓ `User`, `BadgeDefinition`, `BadgeEvent`, `BadgeEventLike`, `BadgeOwnership`,
`ExerciseSet`, `FineRecord`, `GamebookProgress`, `MedicalCertificate`,
`MonthlyChallengeEntry`, `PotEvent`, `WallMessage`

Manquants (déclarés `model X { ... }` mais aucun `CREATE TABLE`) :

✗ `Daemon`
 ✗ `GlobalConfig`
 ✗ `BadgeLike`
 ✗ `UserStatus`
 ✗ `StatusLike`
 ✗ `XpAdjustment`
 ✗ `SpecialWorkoutEntry`
 ✗ `Bet`, `BetEntry`, `BetEvent`, `BetEventLike`, `BetResult`
 ✗ `CoinAdjustment`
 ✗ `QuotaModifier`

Diagnostic : Le schéma de production a été synchronisé via `prisma db push` (sync direct, sans génération de migration) plutôt que `prisma migrate dev`. Conséquences :

- Tout clonage du repo échouera à recréer la prod avec `prisma migrate deploy`.
- Aucune trace d'historique pour ces tables : impossible de rollback, impossible d'auditer l'évolution des contraintes.
- Le `migration_lock.toml` ment sur l'état réel.

Cas le plus critique : `Daemon` — 40 colonnes, FK vers `User`, contient les statpoints, l'XP, l'équipement et l'objet `activeBattle`. C'est la **seconde table la plus critique du Nexus**, et elle n'a pas de migration. Si la prod est cassée demain, la recréation passe par un `db push` manuel.

2.4 Index présents

Table	Index	Coverage
<code>GamebookProgress</code>	<code>@@unique([userId, chapterId])</code> , <code>@@index([userId])</code> , <code>@@index([mapId])</code> , <code>@@index([lastSeen Desc])</code>	Correct pour les requêtes connues
<code>Daemon</code>	<code>@@unique([userId, slotIndex])</code> , <code>@@index([userId])</code>	Correct

Bonne nouvelle : aucun manque d'index sur les hot paths. **Mais** aucun index partiel sur les 42 booléens, ce qui rendra coûteuse toute future analytics (« qui a fini Pastagone ? »).

2.5 ● Intégrité référentielle

Toutes les FK vers `User` ont `onDelete: Cascade`. Cohérent avec la suppression RGPD. **Mais :**

- Aucune FK sortante de `Daemon` ou `GamebookProgress` vers des tables de définition (ex. `BadgeDefinition` pour les badges débloqués). Tous les `badgeKey` sont des **strings libres**. → Impossible de garantir qu'un badge cité dans le payload existe réellement.
- Le champ `chapterId` est un string libre. Aucune table `Chapter` pour le contraindre.

3. État des migrations

3.1 ● Désordre chronologique entre nommage et sémantique

Migrations triées alphabétiquement (= ordre d'exécution Prisma) :

```
20260524_pushquest_v3_3_npcs
20260525_pushquest_v3_4a_energy_spent
20260526_pushquest_v3_6_anti_cheat
20260526_pushquest_v4_0_active_battle          ← v4.0 ici
20260526_pushquest_v4_0_daemon_unlocked
20260526_pushquest_v4_0_pending_stat_points
20260526_pushquest_v4_pastagone_progress
20260527_pushquest_v4_capolino_mid_fuite
20260527_pushquest_v4_pullman_coulter
20260528_pushquest_v3_8_pepiteville           ← v3.8 APRÈS v4.0 ?!
20260529_pushquest_v3_8_1_fruits_boots
20260530_pushquest_v3_8_2_tower
20260601_pushquest_v3_11_piaffini
20260602_pushquest_v3_12_water
20260603_pushquest_v3_14_tamagotchi
20260604_pushquest_v3_17_luck
20260605_pushquest_v3_17c_polish
20260606_pushquest_v3_19b_bestioles
20260607_pushquest_v3_21_casino
20260608_pushquest_v3_22_fast_travel
```

Diagnostic : Les versions sémantiques (v3.8, v3.11, ..., v3.22) ont été créées **après** v4.0. Le timestamp de nommage reflète la date d'écriture, pas l'ordre d'arc narratif. C'est légitime sur le papier — mais ça produit deux problèmes :

1. **Risque de conflit sur colonnes** : si v3.8 ajoute `pepitevilleArrived` et v4.0 a déjà altéré la même table, l'ALTER s'enchaîne dans un ordre incohérent avec le code qui les a introduits.
2. **Lecture humaine impossible** : impossible de reconstituer chronologiquement « quand on a livré quoi » à partir du dossier `migrations/`.

À vérifier (centralisé dans Phase 1.8) : la prod a-t-elle appliqué les migrations dans cet ordre alphabétique, ou un ordre fourni à la main ? Si Vercel a déclenché `migrate deploy` à chaque push, c'est l'ordre alphabétique qui s'applique → les utilisateurs prod ont vécu une « v4.0 active_battle » avant la « v3.8 pepiteville ».

3.2 ● Migrations purement additives — Bon point

Toutes les migrations Nexus sont des `ALTER TABLE ... ADD COLUMN` avec `DEFAULT`. Aucun `DROP COLUMN`, aucun `RENAME`, aucun `ALTER TYPE`. ✅ Cohérent avec la règle d'or énoncée par Sartay (« migrations additives only »).

3.3 ● Migrations qui exécutent du code sur table inexistante (couplé à 2.3)

Plusieurs migrations effectuent des `ALTER TABLE "Daemon" ADD COLUMN ...` alors que la table `Daemon` **n'a jamais été créée par migration**. Sur une base recréée à zéro via `migrate deploy`, ces ALTER plantent. **Inférence** : la branche `feat/gamification-pass-2` n'est aujourd'hui reproductible qu'avec un Neon préexistant ayant subi un `db push` initial.

4. Race conditions & transactions

4.1 ● Zéro transaction sur 95 routes mutatives

```
$ grep -rln "$transaction\|interactiveTransaction" src/app/api/gamebook/
(aucun résultat)
```

95 routes Nexus écrivent en base sans `prisma.$transaction`. Conséquences pratiques sur les scénarios joueur :

Scénario	Tables touchées	Risque
Achat shop	<code>GamebookProgress</code> (énergie) + <code>XpAdjustment</code> (badge) + <code>BadgeOwnership</code>	Énergie débitée mais badge non créé si crash entre 2 writes
Spend énergie + déplacement	<code>GamebookProgress.energySpentToday</code> + <code>posX/posY</code>	Position mise à jour mais énergie non débitée (ou inverse)
Combat Daemon	<code>Daemon</code> (HP, XP) + <code>GamebookProgress.activeBattle</code> (JSON) + <code>XpAdjustment</code>	Daemon level-up sans XP en banque, ou XP créditée 2×
Casino lotto	<code>GamebookProgress.lottoPouleDate</code> + <code>lottoPouleWonToday</code> + <code>XpAdjustment</code>	Joueur peut rejouer en spammant si la 2e écriture rate

4.2 ● 31 routes mutant `energySpentToday`

C'est le champ le plus chaud du Nexus. Chaque écriture suit le pattern :

```
const gbp = await prisma.gamebookProgress.findUnique({...})
const newSpent = gbp.energySpentToday + COST
await prisma.gamebookProgress.update({
  where: { userId_chapterId: {...} },
  data: { energySpentToday: newSpent, ... },
})
```

Anti-pattern read-modify-write sans verrouillage. Si deux requêtes du même joueur arrivent à 100 ms d'intervalle :

1. R1 lit `energySpentToday = 50`
2. R2 lit `energySpentToday = 50`
3. R1 écrit `60` (coût 10)
4. R2 écrit `60` (coût 10) → Le joueur a dépensé 20 d'énergie mais le compteur indique 60 au lieu de 70.

Mitigation existante : le client envoie une action à la fois (`isLoading` du `MapClient`), ce qui réduit la fenêtre.

Mais : multi-onglets, double-clic rapide, retry côté réseau, et surtout les jobs serveur (Pusher events) bypassent cette garantie client.

4.3 ● Aucun verrouillage applicatif

Aucun `pg_advisory_lock`, aucun `SELECT ... FOR UPDATE`. La seule garantie est l'isolation `READ COMMITTED` par défaut de PostgreSQL — insuffisante pour des incréments concurrents.

Pattern correct attendu : soit incrément atomique (`{ increment: COST }`), soit transaction sérialisable avec `retry`, soit advisory lock par `userId`. Aucun des trois n'existe.

4.4 ● Le champ `activeBattle` (JSON) cumule les risques

`Daemon.activeBattle` stocke l'état complet d'un combat (HP, tour, statut). Toute action de combat fait un read-modify-write sur ce JSON :

```
const daemon = await prisma.daemon.findFirst(...)
const battle = daemon.activeBattle as Battle
battle.enemyHp -= damage
await prisma.daemon.update({ where: { id }, data: { activeBattle: battle } })
```

Deux clics rapides « ATTAQUE » → double dégât appliqué côté client mais un seul persisté → désynchro.

5. Inventaire des flags — orphelins & fantômes

5.1 Boolean flags (42)

Tous référencés dans le code, **sauf 2 quasi-orphelins** :

Flag	Référence unique	Statut
<code>durumEnergyGiven</code>	<code>api/gamebook/grant-durum-energy/route.ts</code>	● Set unique, jamais relu pour conditionner
<code>lottoPouleWonToday</code>	<code>api/gamebook/casino/lotto-poule/route.ts</code>	● Set unique, jamais relu

Les deux flags sont écrits par leur route mais **jamais consommés** par une garde de gameplay. Soit la garde a été retirée, soit le flag a été conçu en prévision. **À clarifier** (Phase 1.8).

5.2 Catégorisation thématique des 42 booléens

Thème	Flags	Volume
Onboarding & intro	<code>hasSeenWelcomeScreen</code> , <code>hasBag</code> , <code>hasEnteredTallGrass</code> , <code>monsterCaveRevealed</code> , <code>grassSudCutsceneShown</code>	5
Arc Tour Blagueur (TB)	<code>tbBossBeaten</code> , <code>tbBossKeyHeld</code> , <code>tbRewardClaimed</code> , <code>treeBookGiven</code> , <code>treeObstacleCleared</code>	5
Arc PIAFFINI / Mont	<code>piaffiniRescued</code> , <code>montSummitReached</code> , <code>franssJokeBirdDone</code> , <code>ornithologueBirdBonusGiven</code> , <code>firstSwimDone</code>	5
Muscuville	<code>muscuvilleInterpellatorTalked</code> , <code>muscuvilleRocksPassed</code> , <code>arenaUnlocked</code> , <code>nageurDefiCompleted</code>	4
Contest hall	<code>contestDefiPompatorDone</code> , <code>contestDefiSquatilusDone</code> , <code>contestDefiTiroirDone</code>	3
Pastagone	<code>pastagoneArrested</code> , <code>pastagoneBolognionFound</code> , <code>pastagoneBossBeaten</code> , <code>pastagoneCapolinoFleeShown</code> , <code>pastagoneCapolinoMidBeaten</code> , <code>pastagoneCoulterBeaten</code> , <code>pastagoneEscaped</code> , <code>pastagoneFaaGiftClaimed</code>	8

Thème	Flags	Volume
Casino	<code>bourgCasinoCoinsFound</code> , <code>lottoPouleWonToday</code>	2
PNJ ponctuels	<code>papaBoostClaimed</code> , <code>pereTalked</code> , <code>durumEnergyGiven</code> , <code>gymGuyEnergyGiven</code> , <code>bestiolesFirstEncountered</code>	5
Tamagotchi & jardinier	<code>tamagotchiInBag</code> , <code>jardinierArrosoirGiven</code> , <code>jardinierMissionActive</code>	3
Système	<code>isCompleted</code> , <code>pioneerBadgeAwarded</code>	2

5.3 🟠 « Ghost flags » — utilisés dans le code mais valeur par défaut implicite

Plusieurs flags sont lus en `gbp.xxx ?? false` dans le code mais n'ont pas de `DEFAULT false` explicite dans la migration (vérification rapide nécessaire ligne par ligne — à approfondir en Phase 1.7 / 1.8). Risque : sur un compte créé avant l'ajout du flag, la valeur est `NULL`, le code la traite comme `false`, mais les requêtes SQL ad-hoc retournent du `NULL` au lieu de `false`. Inconsistance silencieuse.

5.4 🟡 Flags JSON `flags` & `history` — fourre-tout

Les deux Json `flags` et `history` sur `GamebookProgress` sont des sacs à clés. Le code y empile des choses sans schéma. Aucun moyen d'auditer ce qui s'y trouve sans grep exhaustif. **Recommandation différée** : à terme, migrer ces clés vers des colonnes typées ou une table EAV `GamebookFlag(userId, key, value)`.

6. Persistance client/serveur

6.1 🟡 Source de vérité dispersée

Donnée	Source de vérité	Stockée côté client ?
Position joueur	<code>GamebookProgress.posX/posY</code>	Oui (state React) — re-fetch sur reload
Énergie restante	Calculée serveur (<code>workouts</code> du jour - <code>energySpentToday</code>)	Oui (état dérivé)
Inventaire	<code>GamebookProgress.inventory</code> (Json)	Oui
HP Daemon	<code>Daemon.currentHp</code>	Oui (<code>activeBattle.actorHp</code> aussi) → 🟠 dualité
<code>activeBattle</code>	<code>Daemon.activeBattle</code> (Json)	Oui (mirror complet)

Risque principal : `Daemon.currentHp` ET `activeBattle.actorHp` représentent la même donnée. Le code lit/écrit l'un OU l'autre selon le contexte. Désynchro garantie si crash en milieu de combat.

6.2 🟠 Aucune trace d'audit côté serveur

Aucune table `GamebookEvent` / `PlayerActionLog` / `AuditTrail`. Si demain un joueur conteste « j'ai pas dépensé cette énergie », il n'existe **aucun journal serveur** pour reconstituer la séquence. Le seul artefact est le JSON `history` (taille non bornée, écrit côté client).

6.3 🟡 Snapshots ?

Aucun mécanisme de snapshot/rollback de `GamebookProgress`. Restaurer un joueur après bug = `UPDATE` manuel via Prisma Studio ou route admin ad-hoc.

7. Top routes par fréquence d'écriture GamebookProgress

Rang	Route	gamebookProgress.update count
1	api/gamebook/state	12
2	api/gamebook/spend	9
3	api/gamebook/daemon/battle/action	9
4	api/gamebook/casino/stop	8
5	api/gamebook/muscuville/rocks-pay	6
6	api/gamebook/shop/buy	5
7	api/gamebook/pastagone/infirmierie-heal	5
8	api/gamebook/inventory/use	4
9	api/gamebook/hotel/sleep	4
10	api/gamebook/franss-joke	4

Diagnostic : la route `/state` (hardénée récemment via commit `78c9d18`) écrit 12 fois — c'est elle qui doit absolument passer en transaction.

8. Synthèse criticité

#	Constat	Criticité	Effort fix
1	God-table 130 champs	●	XXL (refacto domaine)
2	14 modèles sans CREATE TABLE	●	M (régénérer baseline migration)
3	0 transaction sur 95 routes	●	L (audit route par route)
4	31 routes race-prone sur énergie	●	M (incrément atomique)
5	Désordre alphabétique migrations	●	S (renommer ou figer)
6	Dates en <code>String</code>	●	M (migration + code)
7	Dualité <code>Daemon.currentHp</code> / <code>activeBattle.actorHp</code>	●	S
8	Aucun audit log serveur	●	L
9	2 flags quasi-orphelins	●	XS
10	Json fourre-tout <code>flags</code> / <code>history</code>	●	XL

9. Recommandations DBA (sans implémentation)

- Baseline urgente** : générer une migration `__baseline_v3_drift_repair` qui crée explicitement les 14 tables manquantes via `prisma migrate dev --create-only`, puis marquer la migration comme appliquée en prod via `prisma migrate resolve --applied`. Sans ça, le repo est ingérable sur tout nouvel environnement.
- Atomicité énergie** : remplacer tous les read-modify-write `energySpentToday` par `{ increment: COST }`. Bénéfice immédiat, effort minimal, zéro side effect métier.
- Transaction sur routes shopping** : `shop/buy`, `casino/*`, `daemon/battle/action`, `inventory/use` doivent passer en `prisma.$transaction([...])` ou `prisma.$transaction(async tx => ...)`.

- 4. **Journal append-only** : créer table `GamebookEvent(id, userId, eventType, payload Json, createdAt)` et `INSERT` à chaque action significative. Permet rollback, debug joueur, futur analytics.
- 5. **Convention dates** : geler la convention « `*Date` toujours `DateTime`, `*Day` toujours `String YYYY-MM-DD` » et migrer progressivement.

Fin **AUDIT_02_DATABASE.md**.

Phase 1.2 terminée. Les points ● 1, 2, 3, 4, 5 sont les plus alarmants — ils relèvent tous d'un patron systémique (« écriture rapide sans atomicité ») né de la vélocité solo + livraison continue.

Auto Mode actif → j'enchaîne avec **Phase 1.3 : AUDIT_03_GAMEDESIGN.md** (Game Designer) dans la réponse suivante.

<div class="pagebreak"></div>

AUDIT_03_GAMEDESIGN.md — Game Designer

Périmètre : Nexus / Gamebook subsystem
Date : 2026-05-27
Posture : Sans complaisance

1. Boucle de jeu principale

Couche	Élément	Verdict
Métrique d'effort	Reps réelles (pompes, squats, plank/5, etc.)	✓ Excellent : ancrage physique réel
Conversion	10 reps = 1 case (<code>COST_MOVE = 10</code>)	✓ Lisible, mémorisable
Plafond / jour	<code>MAX_SPEND_PER_CALL = 500</code> par appel	● Pas un cap journalier réel — limite par requête seulement
Boucle minute	Move → interaction PNJ/tile → narrative beat	✓ Le bouclage est solide
Boucle session	Faire ses reps du jour → progresser sur la map → croiser un PNJ	✓
Boucle long terme	Compléter arcs (TB, Pastagone, Macaron'île...)	● Voir 1.3

1.1 ● La conversion reps→cases est l'âme du jeu

`COST_MOVE = 10` est non-négociable : c'est ce qui fait que faire des pompes a un coût d'opportunité visible. **À conserver coûte que coûte.**

1.2 ● La progression long terme repose sur des arcs trop nombreux pour un solo

Liste des arcs identifiés via `mapId` + commentaires version :

- v3.0 : Bourg-Boulette / cave / arbre obstacle
- v3.1 : Route 1 / pont Pépite / 4 PNJ
- v3.3 : NPCs Pépiteville
- v3.4 : Bonus surplus, baskets

```

v3.6 : Anti-cheat freeze
v3.8 : Pépiteville (fruits, boots)
v3.11 : PIAFFINI / Tour Blagueur
v3.12 : Water push (mer)
v3.14 : Tamagotchi
v3.17 : Luck / Casino Bourg
v3.19 : Bestioles (hautes herbes sud)
v3.21 : Casino étendu
v3.22 : Fast travel
v3.23 : Mont Pasta-Ventoux / Contest hall
v4.0 : Daemon refactor + Pastagone + Pullman + CAPOLINO + Pierres

```

~15 arcs livrés, chacun avec ses PNJ, ses items, ses flags, ses dialogues. Le périmètre est **gigantesque pour un dev solo** sans testeur dédié. Risque inévitable : surface de bug ingérable (vu en Phase 1.2).

1.3 ● Onboarding sur-codé pour l'effet escompté

Le tutoriel (MONSTER_INTRO_DIALOGUE , 13 étapes + téléport en grotte) prescrit la règle « 10 reps = 1 case » verbalement puis envoie le joueur faire 150 reps pour pousser l'arbre. **C'est 150 pompes avant la première progression.** Pour un joueur peu sportif, c'est un mur. Pour un athlète, c'est insignifiant. Aucune adaptation au profil.

Conséquence : la courbe de difficulté est binaire (« je peux faire 150 pompes » / « je ne peux pas »). Le « tile 1 » n'existe pas — il y a un saut entre le tuto et le premier vrai obstacle.

2. Économie & équilibrage

2.1 ● Stats Daemon — formule très bonne

```

Force      = 25 + totalPushups / 100
Vitesse    = 25 + log10(totalSteps + 1) × 15
Défense    = 25 + 2 × défisValidés
Intelligence = 25 + totalShopSpend / 50
Endurance  = 25 + totalPlankSeconds / 60

```

Clamp [25, 100]. **Lecture de game design** : chaque stat capture une dimension réelle du joueur. Log10 sur les steps évite que les marathoniens dominent. Plancher 25 protège les nouveaux. **C'est probablement la meilleure idée mécanique de tout le Nexus.**

2.2 ● Saiyan : +1 si effort, -1 si farming d'easy → excellent

```

Base = 7, clamp [5, 9]
+1 si energySpentThisLevel ≥ 100
+1 si koCountThisLevel ≥ 1
+1 si hardBattles > easyBattles × 2
-1 si easyBattles > hardBattles × 2 (et hardBattles > 0)

```

Sanctionne le farming, récompense la résilience. **À conserver.**

2.3 ● Courbe XP Pokémon Gen 1 (L³) + cap niveau 50

```

xpForLevel(L) = L3
xpForLevel(50) = 125 000

```

Avec `BASE_EXP_WILD_COMMON = 64` et `computeRewardXp = baseExp × level / 7` :

- L1 vs L1 → ~9 XP/combat → ~**111 combats pour L2** (théorique). En pratique L2 = 8 XP donc 1 combat suffit.
- L25 vs L25 → ~228 XP/combat → ~**68 combats** entre L24 et L25.
- L49 vs L49 → ~448 XP/combat → ~**327 combats** entre L49 et L50.

→ La queue est **interminable**. Si Sartay veut que ses 7 potes voient le L50, il faudrait ~3500 combats pour le dernier niveau seul. C'est probablement **mort** narrativement.

Recommandation : soit cap à L30 (10 fois moins de combats requis), soit baseExp dynamique (boss 4x, rivaux 3x).

2.4 ● Badges tous capés à 200 XP

Décision récente (commit `c8bb83d`) : tous les badges Nexus capés à 200 XP. 5 badges → 1000 XP max sur l'ensemble du jeu. Pour des joueurs qui font 40k XP/an de reps, **les badges sont insignifiants côté XP**. Leur valeur est purement symbolique/collectionnée. ✅ Décision cohérente avec « le jeu n'inflète pas l'XP de l'app PushQuest ».

2.5 ● Casino — économie potentiellement cassée

Sources d'énergie identifiées :

- Reps quotidiennes (variable)
- Bonus surplus (one-shots des PNJ)
- Gains casino (cockfight, lotto-poule, slot, pattern-spin, stop-ou-encore)

5 jeux casino. Tous utilisent `Math.random()` côté serveur. Si l'EV (expected value) > 0 sur même un seul → exploite infini d'énergie. **À vérifier sur chaque jeu** (centralisé en Phase 1.8).

Cap journalier identifié : `casinoBetsToday ≤ 10` pour `bet`, `cockfightDate` pour cockfight, `stopOuEncorePlaysToday`, `slotMachinesPlayedToday`, `lottoPouleDate` (1/j). **Mais** : pas de cap cumulé sur le **gain total** d'un jour. Un joueur chanceux peut multiplier son énergie sans plafond.

2.6 ● Shop — pas de cap de dépense

`totalShopSpend` est tracké (sert à la stat Intelligence) mais aucun cap journalier. Un joueur peut tout dépenser d'un coup ou rien. Pas un bug, mais aucun rythme imposé.

3. Progression & gating

3.1 ● Gating par badges/flags — propre

Chaque arc débloque le suivant via un flag (`piaffiniRescued`, `pastagoneEscaped`, etc.). Lecture simple, idempotent, lisible dans le code.

3.2 ● Gating par défis sportifs — bonne idée, mauvais équilibrage

Pousser l'arbre = 150 reps. Défis Champions de Muscuville = volumes inconnus (à mesurer dans `champion/route.ts`). Le gating crée des **murs déterministes**, mais sans courbe ajustée au profil ça revient au pb du tuto (1.3).

3.3 ● Aucun système de skip / aide / contournement

Pas de menu « j'abandonne cet arc », pas de réduction du coût après N tentatives, pas de hint après un dialogue répété 5 fois. Un joueur bloqué reste bloqué. Vu le bug récent « Gg-rem coincé dans `shop_interior` » : c'est arrivé **en vrai sur la prod**.

4. Combat Daemon

4.1 ● Formule HP Pokémon Gen 1 fidèle

```
maxHp = floor((End + 10) × 2 × Lvl / 100) + Lvl + 10 + bonusEnd
```

4.2 ● Crit cap à 35 % — Cohérent

```
crit = 0.0625 + Int/500 + happiness/1000, max 0.35
```

4.3 ● Bonheur ×0.5 à ×1.5 — Effet visible

```
happinessMultiplier(h) = 1 + (h - 50) / 100
```

4.4 ● Combat = JSON `activeBattle` mutable — voir Phase 1.2 §4.4

Risque de désynchro client/serveur. Pas de game design en soi, mais le combat est **fragile**.

4.5 ● Types implémentés mais nombre limité

`DaemonType` énuméré : Normal, Feu, Eau, Plante, Electrique, Vol, Psy, Pate, Combat, Roche.

Aucune table de faiblesses/résistances visible dans `daemon.ts`. À vérifier dans `combat.ts` ou `attacks.ts` (Phase 1.8).

5. Items & inventaire

Catégorie	Count
Items totaux	48
Wearables (baskets, brassards, lunettes...)	~10 (passage rapide)
Consommables	~15
Stockables (gourde, bidon)	2
Cosmétiques	~5

5.1 ● Wearables à durabilité — bon design

Baskets réduisent le coût de déplacement de 10→8 pendant 500 pas. Trade-off lisible.

5.2 ● 48 items pour 7 joueurs — surface trop grande

Probablement trop d'items pour qu'un joueur les goûte tous. Phénomène classique de jeu solo. Pas critique, mais 30 % des items doivent être rarement vus.

6. Synthèse criticité

#	Constat	Criticité
1	Onboarding = mur 150 reps non gradué	●
2	Courbe L ³ + cap L50 = queue interminable	●

#	Constat	Criticité
3	Casino sans cap de gain journalier cumulé	●
4	Aucun système de skip/contournement	●
5	15 arcs pour un dev solo + 7 joueurs	●
6	48 items, dilution attention	●
7	Pas de cap shop journalier	●

<div class="pagebreak"></div>

AUDIT_04_NARRATIF.md — Narrative Designer

Périmètre : Nexus / Gamebook subsystem

Date : 2026-05-27

1. Architecture narrative

1.1 Personnages identifiés

105 PNJ totaux (count `id` dans `npcs.ts`).

Personnages-piliers :

- Monstre Spaghetti Volant (mentor irrévérencieux, intro)
- MAMAN / PEPITO (donneurs de sac/baskets)
- JOJO / JOJETTE / PIAFFINI (arc Tour Blagueur → sauvetage oiseau)
- CAPOLINO (rival, 4 rencontres)
- BOLOGNION (secret, mafia Team Boulette)
- Asriel / Coulter / Faa / Coram (Pullman PastaVegas, 4 étages)
- Roger / Lyra / Will / Lee (lore Pullman)
- Mary Malone / lorek / Lee Scoresby / Serafina Pekkala (Pullman, donneurs)
- Champions Muscuville (Pompator / Squatilus / Tiroir + revanche)

Diagnostic : 4 univers de référence se croisent (Pokémon Gen 1, Dragon Ball / Saiyan, Pullman *His Dark Materials*, Pasta humor). C’est riche, **mais** :

1.2 ● Fusion d’univers non assumée

Pullman (sérieux/mystique) + blagues de pâtes + Pokémon → ton oscillant. Le Monstre Spaghetti Volant brise le 4e mur (« arrête de me regarder comme ça ») cohabite avec Asriel / lorek (références fidèles à un univers tragique). **Aucun joueur n’a verbalisé la dissonance pour le moment, mais c’est un risque de cohésion.**

2. Dialogues

2.1 ● Ton du Monstre Spaghetti Volant — excellent

Direct, fainéant, paternel, drôle. C'est **la voix forte** du jeu :

« 10 reps = 1 case. Pas plus, pas moins. »
 « Va te faire des pompes avant. »
 « Et arrête de me regarder comme ça. »

2.2 ● PIAFFINI / Franss-joke — cassure 4e mur réussie

Le rappel pour blaguer « j'avais oublié un truc → AHAHAH c'était une blague » est exactement le registre d'une amitié geek bienveillante (clin d'œil au Dofus de Sartay, je suppose). À conserver.

2.3 ● Dialogues conditionnels — gestion fragile

PEPITO_DIALOGUE_FIRST vs PEPITO_DIALOGUE_AFTER : pattern clair. **Mais** : v3.22 réutilise le même dialogue pour MAMAN OU PEPITO selon la branche du joueur. Le texte parle de « *Te tend un sac* » et « *Sort une paire de baskets* » — pour MAMAN c'est cohérent, pour PEPITO aussi. ✓ Bon réemploi.

2.4 ● TOWER_JOKES — humour à 100 % spécifique

"Pourquoi est-ce que les pâtes sont sportives ?"
 "Parce qu'elles ont la forme."

4 blagues sur les pâtes. Drôle 1 fois, lassant à la 10e visite. **Aucun gate pour éviter le replay.** À mettre dans Phase 1.5 (UX).

2.5 ● Aucune table centrale de dialogues

dialogue.ts contient ~10 dialogues système (intro, pioneer, pepito, piaffini, jojo, franss-joke, mont, rumeurs, tower jokes). **Tous les autres dialogues** (PNJ ponctuels, champions, mafiosi, jardinier) sont éparpillés dans npcs.ts (2565 lignes) ou inline dans les routes.

→ Impossible de relire la narration d'un arc en un endroit unique. Tout audit éditorial (cohérence du « tu » / « vous », orthographe, ton) nécessite de scanner 5+ fichiers.

3. Arcs narratifs

3.1 Inventaire des arcs (par mapId + flags)

Arc	Trigger	Sortie	Récompense	Statut
Intro	Spawn bourgpates	treeObstacleCleared	—	✓ Live
Pionnier	Pousser arbre	pioneerBadgeAwarded	Badge 200 XP	✓ Live
Pont Pépite	4 PNJ Route 1	Accès pepiteville	—	✓ Live
Tour Blagueur	tower_floor_1→4	piaffiniRescued	Set de Nage	✓ Live
PIAFFINI rescue	Sommet tour	Badge Sauveur 200 XP	+ dialogue JOJO	✓ Live

Arc	Trigger	Sortie	Récompense	Statut
Mer	Set de Nage équipé	<code>firstSwimDone</code>	Accès sud	✓ Live
Mont Pasta-Ventoux	Atteindre sommet	Badge Conquérant 200 XP	+ Muscuville/Contest	✓ Live
Muscuville	3 défis Champions	Badge Champio Star (revanche)	200 XP	✓ Live
Contest Hall	3 défis	<code>contestDefiXxxDone</code>	XP cumulé	✓ Live
Bestioles	Hautes herbes sud	<code>bestiolesFirstEncountered</code>	Bestiaire	✓ Live
Tamagotchi	v3.14	Daemon migration	Pierres / Sérums	✓ Live
Casino Bourg	Coin trouvée	<code>bourgCasinoCoinsFound</code>	Casseur de banque	✓ Live
Pastagone	Arc complet	<code>pastagoneEscaped</code> + <code>pastagoneBossBeaten</code>	Bolognion / Pierres	✓ Live
Pullman PastaVegas	4 étages	Coulter mini-boss	Wearables / Sérums	✓ Live
CAPOLINO rival	4 rencontres	Combat boss puis fuite	Lore	✓ Live
Macaron'île	Serafina	Sérums boost permanent	XP	✓ Live
Fast travel	v3.22	Téléport entre villes	—	✓ Live

3.2 ● Aucune linéarisation explicite

Un joueur ne sait pas **dans quel ordre** faire les arcs. Le seul guide est « j'ai débloqué tel flag, donc je peux maintenant aller là ». Avec 17 maplds et ~15 arcs, c'est désorientant.

3.3 ● Arcs sans clôture émotionnelle

Plusieurs arcs débloquent un badge puis... rien. PIAFFINI a une vraie clôture (dialogue JOJETTE + cadeau). Mont Pasta-Ventoux a une voix-off sommet. **Pastagone** se termine sur quoi ? À vérifier (Phase 1.8). Si un joueur traverse Pastagone et ne reçoit pas un beat narratif fort à la sortie, c'est anti-climactique.

3.4 ● Foreshadowing présent

`RUMEUR_OISEAU_DIALOGUE` , `RUMEUR_HERBES_DIALOGUE` , `RUMEUR_CONCOURS_DIALOGUE` plantent PIAFFINI, bestioles, contest. ✓ Bonne pratique de game writing.

4. Items narratifs

4.1 ● Items avec lore

- Gourde, Bidon d'IL CAPO (trophée)

- Set de Nage (héritage grand-mère JOJO)
- Tenue VIP casino
- Pierres d'évolution (Lee Scoresby)
- Sérums (Serafina, boost permanent)

Chaque item a sa raison narrative d'exister. 

5. Synthèse criticité

#	Constat	Criticité
1	Aucune table centrale dialogues	
2	Aucune linéarisation explicite des arcs	
3	Fusion d'univers (Pullman + pasta) non assumée	
4	Plusieurs arcs sans clôture émotionnelle	
5	TOWER_JOKES sans gate replay	
6	Foreshadowing fait	

<div class="pagebreak"> </div>

AUDIT_05_UX.md — UX Designer

Périmètre : Nexus / Gamebook subsystem

Date : 2026-05-27

1. Architecture UX

1.1 Le `MapClient` est l'unique écran

Tout passe par `MapClient.tsx` (5791 lignes) :

- Affichage map + sprites
- HUD énergie
- Modales (dialogue, shop, inventaire, combat, casino, défi)
- Cinématiques (téléport, sommet, sauvetage)

→ **Un seul écran, beaucoup de couches modales empilées.** Aucune route URL distincte pour chaque sous-écran. Conséquence : F5 perd l'état modal local (un joueur en plein dialogue qui recharge la page revient au monde).

1.2 **Aucun deeplink**

Impossible d'envoyer à un pote « va voir ce dialogue ici » ou de bookmarker une situation. Toute la navigation est state-only.

2. HUD & feedback

2.1 ● HUD énergie permanent — bon

L'énergie disponible est toujours visible. Le coût d'un déplacement (10) est implicite mais constant.

2.2 ● Pas de prévisualisation du coût

Aucune surimpression « -10 » au survol d'une case. Le joueur apprend par l'expérience. OK pour le tuto, mais sur une décision de longue distance (« si je vais à Pépiteville, ça me coûte combien ? »), pas d'info anticipée.

2.3 ● Feedback du débit pas toujours synchrone

Plusieurs routes `update` puis le client re-fetch. Latence visible (250-600ms). Pas d'optimistic UI cohérent. Sur les actions rapides (combat, casino), le joueur voit son HP/énergie sauter en différé.

2.4 ● Aucune notification persistante

Pas de toast d'historique « +200 XP Pionnier reçu ». Le dialogue passe une fois, après c'est perdu. Pas de panneau « événements de la session ».

3. Onboarding

3.1 ● Pas de tutoriel des contrôles

Le joueur arrive sur le `MapClient`. Aucun overlay ne lui dit « flèches pour bouger », « ESC pour menu », « E pour interagir », « START pour ouvrir le sac ». Le dialogue d'intro mentionne « START pour ouvrir ton sac » au milieu d'un texte sans mise en évidence visuelle du bouton.

3.2 ● Le `hasSeenWelcomeScreen` est binaire

Soit le joueur a vu, soit pas. Aucune re-onboarding « ça fait 2 mois, voici ce que tu as oublié ». 7 potes ≠ tous régulier.

4. Modalité dialogue

4.1 ● Avancement step-by-step lisible

L'utilisateur clique pour avancer. Cohérent avec les JRPG.

4.2 ● Aucun back/forward

Un joueur qui clique trop vite et rate une ligne ne peut pas revenir. Sur des intros de 13 lignes, c'est frustrant.

4.3 ● Pas de skip pour dialogues déjà vus

Re-visiter PEPITO joue toujours `PEPITO_DIALOGUE_AFTER` (2 lignes — OK), mais d'autres PNJ rejouent toute leur intro. À vérifier (Phase 1.8).

5. Combat Daemon

5.1 ● Aucune indication de type

Les 10 types Daemon (Feu, Eau, Plante, Combat, Roche...) existent en code. Le joueur les voit-il ? Avec quel feedback ? Si non → frustration garantie au moment d'un combat où le type compte.

5.2 ● Aucune UI de “vue d’équipe”

`Daemon.slotIndex` suggère plusieurs slots. La gestion d’équipe (swap, ordre, équipement par daemon) doit avoir une UI. **À cartographier en Phase 1.8.**

6. Casino

6.1 ● 5 jeux distincts visuellement différents

Bonne variété (rouge/noir, cockfight, lotto, slot, pattern, stop-ou-encore).

6.2 ● Aucune mémoire des stats joueur

Le joueur ne voit pas « tu as gagné X / perdu Y depuis le début ». Pas de feedback méta. Une simple ligne dans l’inventaire suffirait.

7. Accessibilité

7.1 ● Aucune option contraste/taille texte

Les dialogues sont sur fond foncé (à vérifier), texte petit sur mobile. Aucune option d’accessibilité visible dans le code.

7.2 ● Aucune nav clavier exhaustive

À vérifier — `MapClient` semble utiliser les flèches pour bouger, mais les modales (sac, shop) sont-elles entièrement navigables au clavier ?

7.3 ● Aucune annonce aria

Le mode lecteur d’écran est probablement inutilisable.

8. Mobile

8.1 ● Pas de contrôles tactiles évidents

Le code de `MapClient` mérite un audit visuel. Sartay teste-t-il mobile ?

9. Synthèse criticité

#	Constat	Criticité
1	Aucun deeplink / routing modal	●
2	Pas de tuto des contrôles	●
3	Pas de back/forward dialogue	●
4	Pas d’UI équipe Daemon	●
5	Pas d’options d’accessibilité	●
6	Pas de prévisualisation coût	●
7	Feedback débit non optimiste	●

#	Constat	Criticité
8	Aucune mémoire stats casino	●
9	Mobile non audité	●

<div class="pagebreak"></div>

AUDIT_06_QA_BUGS.md — QA Engineer

Périmètre : Nexus / Gamebook subsystem

Date : 2026-05-27

1. Bugs connus / récents (extraits commit log)

Commit	Bug	Statut
b9a28cb	shop_interior spawn sur sprite NUTRIPATES	✓ Fixé
b9a28cb	MAMAN demande 2 talks pour le sac	✓ Fixé
b9a28cb	Véto cages visual	✓ Fixé
b9a28cb	"recovered gate"	✓ Fixé
706f20e	Revanche Champion +800 XP annoncé mais 0 distribué	✓ Fixé (capé à 200)
78c9d18	POST /state acceptait positions sur tiles bloquantes	✓ Fixé (hardening)
71b972d	Portes (entry positions + exitTargets) incohérences	✓ Fixé
Mémoire	Bonus minuit (pommiers, papa, capitaine...) perdus à minuit	● Non fixé
Phase 1.2	Race condition energySpentToday (31 routes)	● Non fixé

1.1 ● Le bug "bonus minuit" est documenté en mémoire mais pas planifié

Mémo Sartay : « les bonus gagnés (pommiers, papa, capitaine...) sont perdus à minuit. À fixer plus tard. » → **risque de frustration récurrente**. Si un joueur claim un bonus à 23h57, il perd tout à 00h00. Probable cause : un reset journalier touche `bonusSurplus` ou un flag de date confondu.

1.2 ● 31 routes race-prone (cf. Phase 1.2 §4.2)

Symptôme observé concrètement : impossible à reproduire facilement, mais sur 7 joueurs en parallèle qui spam-cliquent, statistiquement il y a déjà eu des désynchronos d'énergie. Aucune trace serveur pour le détecter.

2. Edge cases identifiés à risque

2.1 ● Reset journalier (minuit Paris)

11 champs stockent une date `String` (voir Phase 1.2 §2.2). À chaque rollover minuit, le code doit savoir :

- Reset `energySpentToday` ?

- Reset `casinoBetsToday` ?
- Reset `bonusSurplus` ?
- Reset `casinoBoostPctToday` ?
- Reset `slotMachinesPlayedToday` ?
- Reset `stopOuEncorePlaysToday` ?

Chaque route fait son propre check `if (storedDate !== today) reset`. **Aucune source unique de vérité.**
 Risque : oublier un reset, ou en faire deux à des moments différents (cf. bug bonus minuit).

2.2 ● Concurrency multi-onglets

Un joueur avec 2 onglets ouverts → 2 sessions partageant la même DB. Aucune protection. Si onglet A spend 50 et onglet B spend 50 simultanément, énergie peut sauter à -10 ou rester à 100 (selon ordering DB).

2.3 ● Crash en plein combat = état `activeBattle` orphelin

`Daemon.activeBattle` est un Json. Si le client crash après le serveur a écrit l'init du combat mais avant la première action, le joueur revient et son daemon est "en combat" sans interface visible. **Aucune route de reset visible.**

2.4 ● Position bloquante après hardening

Le commit `78c9d18` empêche d'ÉCRIRE une position bloquante. Mais que se passe-t-il pour un joueur dont la position EN BASE était déjà bloquante avant le patch ? → Le `state` GET retourne la position bloquante, le client tente de bouger, le POST refuse, infinite loop possible. **À vérifier.**

2.5 ● `pioneerBadgeAwarded` et arc Pionnier

Si `treeObstacleCleared = true` mais `pioneerBadgeAwarded = false` (par exemple si le commit de badge a failed), le joueur a passé l'arbre mais n'a jamais reçu le badge. Aucune route de re-claim n'est apparente.

3. Robustesse des routes

3.1 ● 26 routes ont un `console.error` ou `throw`

Sur 95 routes, ~26 loggent ou throw. Les 69 autres : soit succèdent silencieusement, soit catch silencieusement (voir Phase 1.1 : 31 silent catches). **Couverture d'erreur faible.**

3.2 ● Aucun test unitaire visible

Aucun fichier `*.test.ts` ou `*.spec.ts` dans `src/lib/gamebook/`. Les formules critiques (`xpForLevel`, `computeMaxHp`, `computeSaiyanPoints`, `computeDaemonBaseStats`) n'ont aucune validation automatisée. Un changement de constante peut casser le balancing sans alerte.

3.3 ● Aucun test E2E

Aucun Playwright/Cypress trace. Un dev solo en prod sans tests = chaque déploiement est un coup de poker.

4. Données corrompues récurrentes

4.1 ● `flags` Json fourre-tout

Aucun schéma. Toute clé peut être écrite. Si une route v3.8 écrit `flags.fooBar = true` puis le code de v4.0 ne le relit jamais, la clé persiste indéfiniment. **Aucun nettoyage.**

4.2 **history** **Json non borné**

Probable cause : croissance illimitée. À vérifier qu'il n'y a pas de joueur avec 50 KB de history.

5. Synthèse criticité

#	Bug/Risque	Criticité	Statut
1	Bonus minuit perdus		Non fixé
2	31 routes race-prone		Non fixé
3	Reset journalier dispersé		Latent
4	Concurrence multi-onglets		Latent
5	Aucun test unitaire		Latent
6	Aucun test E2E		Latent
7	Crash combat = état orphelin		Latent
8	Position bloquante héritée		À vérifier
9	Pionnier badge potentiellement loupé		Latent
10	Flags Json non purgés		Latent

<div class="pagebreak"></div>

AUDIT_07_CONFORMITE.md — Conformité / Sécurité / Anti-cheat

Périmètre : Nexus / Gamebook subsystem

Date : 2026-05-27

1. Authentification

1.1 **NextAuth sur toutes les routes**

Chaque route mutative commence par `getSession(authOptions)` puis vérifie `session?.user?.id`. **Cohérent.**

1.2 **Aucun rate limiting**

Aucun middleware visible (`next.config / middleware.ts` à vérifier). Un attaquant authentifié peut spam-poster 1000 req/s sur `/spend`. Pas critique en dev, **dangereux en prod si quelqu'un voulait nuker l'énergie d'un pote** (impossible sans son auth) ou DDOS le Neon.

1.3 ● Pas de CSRF check explicite

NextAuth gère le CSRF pour ses propres routes mais les routes `/api/gamebook/*` ne semblent pas avoir de validation CSRF token explicite. À vérifier (Phase 1.8) — NextAuth peut le gérer transparently via cookie `SameSite=Lax`.

2. Autorisations

2.1 ● Le `userId` vient toujours de la session

Aucune route ne lit un `userId` depuis le body de la requête. ✓ Pas d'IDOR (Insecure Direct Object Reference).

2.2 ● `isCreator` / `isSystem` backdoors

`src/lib/gamebook/creator.ts` :

- `padAvailableEnergyForCreator` → énergie min 1000 pour `isSystem=true`
- `bridge/route.ts` : bypass tous les checks de défi
- `spend/route.ts` : `availableEnergy = Math.max(availableEnergy, 1000)`

111 références à `isCreator/isSystem` dans les routes Nexus. Toute personne avec `isSystem=true` en DB a des super-pouvoirs. **Si ce flag est jamais accidentellement activé sur un compte joueur, le balancing est cassé.**

→ Aligné avec Partie B du brief (compte test God Mode externe). C'est exactement ce que la Partie B veut remplacer.

2.3 ● `isSystem` exclu de classements

`bridge/route.ts:296` filtre `user: { isSystem: false }`. ✓ Bonne pratique pour ne pas polluer les leaderboards.

3. Anti-cheat

3.1 ● Freeze gamebook si suppression de reps (v3.6)

`isGamebookFrozen` + `gamebookFrozenUntil` : si un joueur supprime ses reps après avoir spend de l'énergie, le gamebook est gelé. ✓ Idée correcte.

3.2 ● La freeze n'est pas réversible côté joueur

Aucune route `/api/gamebook/unfreeze` côté joueur. Le créateur peut update en DB. Pour 7 potes, c'est gérable. **Mais** : si un joueur a 3-4 freeze par mois, son XP narrative s'éteint.

3.3 ● Aucune validation cryptographique des actions

Le client envoie « j'ai cliqué sur PNJ X, je veux le défi ». Le serveur fait confiance. Si un joueur ouvre la console et POST directement `/api/gamebook/piaffini/rescue`, est-ce qu'il rescue PIAFFINI sans avoir fait la tour ? **À vérifier (Phase 1.8)**. Probablement oui pour certaines routes.

3.4 ● Aucun audit log des actions sensibles

Cf. Phase 1.2 §6.2. Si un joueur conteste ou si un autre est suspect de triche, **aucune trace serveur** pour départager.

4. RGD

4.1 Cascade delete

`onDelete`: `Cascade` sur toutes les FK vers `User` .  Conforme à l’effacement RGPD.

4.2 Aucune export/portabilité

Aucune route `/api/user/export` visible. Si un joueur demande son data, c’est manuel.

4.3 Aucune mention CGU/CGV in-game

Pas de gate « j’ai lu les conditions ». Pour 7 potes, OK. En cas d’élargissement → à prévoir.

5. Secrets

5.1 `.env` présent dans le repo (vu dans `ls`)

Le `.env` est dans le repo (`.gitignore` à vérifier). Si le fichier réel est tracké, c’est un leak. À confirmer (Phase 1.8).

5.2 `.env.example` séparé

Bonne pratique vue.

6. Conformité business

6.1 Casino simulé sans argent réel

Pas de PSC, pas de question légale en France (pas de jeu d’argent au sens du code monétaire).

7. Synthèse criticité

#	Constat	Criticité
1	Pas de rate limiting	
2	CSRF à vérifier	
3	111 backdoors <code>isCreator</code>	 (= raison d’être de Partie B)
4	Validation actions sensibles côté serveur incomplète	
5	Aucun audit log	
6	Aucune export RGPD	
7	<code>.env</code> tracké ?	

<div class="pagebreak"></div>

AUDIT_08_QUESTIONS.md — Questions ouvertes (centralisé)

Pour chaque question : contexte + impact si non répondue.

1. Migrations & DB

Q1.1 — Ordre d'application des migrations en prod

Contexte : v3.8/v3.11/v3.12 datées 20260528+ viennent alphabétiquement APRÈS v4.0 datées 20260526.

Question : Sur la prod Neon, dans quel ordre ces migrations ont-elles été appliquées ? Ordre alphabétique (Prisma standard), ordre manuel via `db push`, ou hybride ?

Impact : Détermine si les utilisateurs prod ont vu une « v4.0 active_battle » avant les colonnes v3.8 Pépiteville, ou l'inverse.

Q1.2 — Régénération depuis zéro

Contexte : 14 modèles sans `CREATE TABLE`.

Question : Faut-il accepter de générer une migration « baseline_drift_repair » via `prisma migrate dev --create-only` puis la marquer `--applied` en prod ? Ou laisser le repo non-reproductible ?

Q1.3 — Bonus minuit

Contexte : Bug documenté en mémoire.

Question : Quels champs exactement sont perdus à minuit ? Lesquels doivent persister (bonus permanent) vs reset (énergie quotidienne) ?

2. Game design

Q2.1 — Cap niveau Daemon

Contexte : Cap $L50 + L^3 \rightarrow$ queue impossible.

Question : Cap à 30 ? Ou baseExp dynamique multiplié pour boss ?

Q2.2 — Casino EV

Contexte : 5 jeux casino, tous en `Math.random()` serveur. Aucun cap de gain cumulé journalier.

Question : EV exact de chaque jeu ? L'un d'eux a-t-il $EV > 0$?

Q2.3 — Pénalité freeze

Contexte : `gamebookFrozenUntil` actif si suppression de reps. Pas de route de réversion joueur.

Question : Garder le freeze comme dissuasion ou ajouter un « rachat » via un défi sportif ?

Q2.4 — Système de skip

Contexte : Joueur bloqué $\rightarrow$ bloqué.

Question : Ajouter un système d'aide après N tentatives ? Sous quelle forme ?

3. Narratif

Q3.1 — Clôture Pastagone

Contexte : Arc majeur v4.0.

Question : Beat narratif post-Pastagone ? Qui dit quoi quand le joueur sort ?

Q3.2 — Fusion univers

Contexte : Pullman + Pasta humor + Pokémon mécanique.

Question : Assumer pleinement le mélange (in-jokes croisés) ou re-cloisonner les tonalités par zone ?

Q3.3 — Linéarisation explicite

Contexte : 15 arcs, ordre flou.

Question : Ajouter une carte/journal de quêtes ? Ou rester sur la découverte organique ?

Q3.4 — TOWER_JOKES replay

Contexte : 4 blagues qui rejouent indéfiniment.

Question : Ajouter un flag `tbJokesSeenIds: int[]` pour ne pas proposer les blagues vues ?

4. UX

Q4.1 — Tutoriel contrôles

Question : Un overlay au premier `MapClient` mount ?

Q4.2 — Back/forward dialogue

Question : Comportement attendu : `<` pour reculer d'1 step ?

Q4.3 — Mobile

Question : Le jeu est-il testé sur mobile ? Sartay utilise-t-il son téléphone pour jouer ?

Q4.4 — UI équipe Daemon

Question : Existe-t-il une UI pour gérer plusieurs Daemons ? Si non, c'est prioritaire ?

5. QA / robustesse

Q5.1 — Tests

Question : Acceptes-tu d'introduire un fichier `daemon.test.ts` minimal (formules) en mode « pas casser le code mais ajouter sécurité » ?

Q5.2 — Position bloquante héritée

Question : Faut-il une route admin de « rescue all blocked » à exécuter une fois ?

Q5.3 — Crash combat orphelin

Question : `activeBattle = null` au timeout (>30min sans action) ?

6. Sécurité

Q6.1 — CSRF

Question : Confirmer que NextAuth gère bien le CSRF via cookie sur les routes `/api/gamebook/*` ?

Q6.2 — Validation actions sensibles

Question : Faut-il auditer route par route les pré-conditions serveur (« le joueur doit avoir tel flag pour POST telle action ») ?

Q6.3 — .env tracké

Question : Vérifier `git ls-files .env`. Si tracké, à purger immédiatement de l'historique ?

Q6.4 — Rate limit

Question : Acceptes-tu un middleware Next.js rate-limit (ex: Upstash) ou ce n'est pas prioritaire pour 7 potes ?

7. Partie B (God Mode)

Q7.1 — Périmètre panel testeur

Question : Quelles fonctions précises veux-tu (resources, time-travel, flag manip, téléport, snapshots) ? Doit-on inclure la simulation de race conditions ?

Q7.2 — Lien avec `isSystem`

Contexte : 111 backdoors actuels.

Question : Le compte test doit-il rester `isSystem=false` (= joueur normal) et le panel agir via routes admin séparées ? Confirme.

```
<div class="pagebreak"></div>
```

```
<a id="10"></a>
```

AUDIT_00_SYNTHESE.md — Synthèse exécutive

Périmètre : Nexus / Gamebook subsystem

Date : 2026-05-27

Branche audité : `feat/gamification-pass-2` HEAD `6f0bb65`

1. Verdict global

Le Nexus est un produit live, joué par 7 vrais utilisateurs, avec une vélocité de production solo remarquable. Il fonctionne. Mais il porte une **dette technique systémique** née de la livraison continue sans tests, sans transactions, et avec une god-table de 130 colonnes.

Dimension	Note	Commentaire
Code	C+	Architecture étroite (MapClient 5791l, npcs 2565l, maps 2812l), 292 <code>(prisma as any)</code> , 0 tests. Cohérent en surface, fragile en profondeur.

Dimension	Note	Commentaire
DB	D	God-table, 14 modèles sans migration, 0 transaction, 31 routes race-prone.
Game design	B	Idées fortes (10 reps = 1 case, formules Daemon, Saiyan). Casino non équilibré, courbe L ³ impraticable.
Narratif	B	Voix forte du Monstre, foreshadowing, mais dispersion et pas de linéarisation.
UX	C	Modale-only, pas de tuto contrôles, pas de deeplink, mobile non confirmé.
QA	D	Aucun test, bugs latents (race, minuit, multi-onglets).
Conformité	C+	NextAuth OK, mais 111 backdoors <code>isCreator</code> , pas d'audit log, pas de rate limit.

2. Top 10 priorités (toutes phases confondues)

Rang	Constat	Phase	Criticité	Effort
1	31 routes race-prone sur <code>energySpentToday</code> (read-modify-write sans tx)	1.2	●	M
2	0 transaction Prisma sur 95 routes	1.2	●	L
3	14 modèles sans <code>CREATE TABLE</code> (schema drift, repo non-reproductible)	1.2	●	M
4	Bonus minuit perdus (bug documenté, non fixé)	1.6	●	S
5	Courbe XP L³ + cap L50 = injouable jusqu'au bout	1.3	●	S (constants)
6	Casino sans cap de gain cumulé journalier	1.3	●	S
7	Aucun test unitaire/E2E	1.6	●	L (mais L = sécurité long terme)
8	Aucun audit log serveur	1.2 / 1.7	●	M
9	MapClient 5791 lignes (god component)	1.1	●	XL
10	111 backdoors <code>isCreator</code> → à remplacer par panneau testeur externe (Partie B)	1.7	●	M (Partie B)

3. Schéma de causalité

```

Vélocité solo + livraison continue
↓
Dette transactionnelle (0 tx, 31 race-prone)
↓
Couplage god-table (130 champs)
↓
Architecture client god-component (MapClient 5791l)
↓
Aucun test
↓
Régressions cycliques (cf. commits b9a28cb, 706f20e, 78c9d18)

```

Chaque couche cristallise les choix de vitesse. Aucune n'est « fautive » isolément — c'est le **système** qui est sous-tendu.

4. Recommandation stratégique

4.1 Priorité absolue (avant tout nouveau contenu)

1. **Baseline migration de répare** (Phase 1.2 §2.3) — sans ça, le repo est inutilisable hors prod actuelle.
2. **Atomicité énergie** (incrément Prisma) — fix systémique, faible effort, haut impact.
3. **Audit log serveur** (`GamebookEvent` append-only) — débloque debug, conformité et anti-cheat.
4. **Fix bug bonus minuit** — frustration utilisateur réelle.

4.2 Stabilisation moyen terme

5. **Suite de tests minimale** : formules Daemon + 5 routes critiques en intégration.
6. **Rééquilibrage L³** (cap L30 ou baseExp boss×4).
7. **Cap casino journalier** sur gain cumulé.

4.3 Refactor long terme (NON BLOQUANT pour l'audit)

8. **Décomposer MapClient** en hooks (cf. Phase 1.1).
9. **Séparer god-table** : `GamebookCore` (position, énergie) + `GamebookFlags` + `GamebookCasino`.
10. **Centraliser dialogues** dans une table de référence éditable.

4.4 Partie B — God Mode externe

Le brief Partie B (panneau testeur séparé, compte normal sans `isSystem`) est **précisément l'antidote** aux 111 backdoors actuels. Recommandation : remplacer progressivement les `isCreator` par des routes `/api/admin/*` cookie-gated par un secret, accessibles uniquement depuis le panel.

5. Ce qui est solide (à NE PAS toucher)

Sans complaisance mais sans non-plus jeter le bébé :

- `COST_MOVE = 10` : intouchable.
- **Formules** `computeDaemonBaseStats` & `computeSaiyanPoints` : à conserver telles quelles.
- **NextAuth** + `userId` depuis `session` : pattern sain.
- `onDelete` : **Cascade partout** : conforme RGPD.
- `isSystem` **exclu des classements** : bonne discipline.
- **Foreshadowing narratif** : à étendre.
- **Voix du Monstre Spaghetti Volant** : marque identitaire.

6. Coût estimé (ordres de grandeur, sans engagement)

Bloc	Estimation Sartay seul
Baseline migration	2-4h
Atomicité énergie sur 31 routes	8-16h
Audit log	4-8h

Bloc	Estimation Sartay seul
Fix bonus minuit	1-3h
Tests formules Daemon	2-4h
Cap L30 ou rebalance	1-2h
Cap casino journalier	2-4h
Décompo MapClient	30-60h
Refactor god-table	40-80h
Partie B (panel testeur)	12-24h

Travail prio (avant nouveau contenu) : ~40h.

Refactor long terme : ~120h (donc à étaler).

7. Ce qui dépend de réponses (Phase 1.8)

22 questions ouvertes centralisées en `AUDIT_08_QUESTIONS.md`. Les plus urgentes :

- Q1.1 (ordre migrations prod)
- Q2.1 (cap L30 ?)
- Q2.2 (EV casino par jeu)
- Q7.1 / Q7.2 (périmètre Partie B)

Fin AUDIT_00_SYNTHESE.md — Fin de la Partie A.